

TriGAC: Graph-Based Interactive Multi-View Clustering with Active Constraints

Xun Fu¹, Lanting Fang¹, Kaiyu Feng¹, Jie Gui², Shuliang Wang¹

1. School of Computer Science, Beijing Institute of Technology

2. School of Cyber Science and Engineering, Southeast University

fuxun0529@163.com; ltfang@bit.edu.cn; fengky@bit.edu.cn; guijie@seu.edu.cn; slwang2011@bit.edu.cn

Abstract—Real-world data are often described by multiple views or modalities, providing complementary information for clustering. Although multi-view clustering has been extensively studied, most existing methods are fully unsupervised and cannot effectively exploit supervisory signals such as pairwise constraints. Existing semi-supervised methods either assume pre-given constraints or rely on costly global optimization, making them less suitable for scalable human-in-the-loop interaction. To address these challenges, we propose Triple Graph Active Clustering (TriGAC), a unified graph-based framework for scalable semi-supervised multi-view clustering with active constraints. TriGAC jointly maintains three lightweight graph structures: object graphs capture view-specific multi-granularity structures, a constraint graph stores actively acquired supervision, and a view graph models cross-view consistency for adaptive weighting. Through their closed-loop interaction, TriGAC progressively refines the consensus partition while avoiding dense constraint matrices and repeated global optimization. Extensive experiments on 15 real-world datasets demonstrate that TriGAC consistently outperforms state-of-the-art semi-supervised multi-view clustering methods in both clustering quality and computational efficiency.

Index Terms—Scalable Clustering, Semi-supervised Clustering, Multi-view Learning, Active Learning, Pairwise Constraint

I. INTRODUCTION

Multi-view clustering is a fundamental task in data mining, aiming to discover meaningful cluster structures by leveraging complementary information from multiple views [1], [2]. High-quality clustering is critical in many applications, as different views often provide distinct perspectives of the same data, and capturing their complementarity can significantly improve clustering utility [3], [4]. However, recovering the underlying ground-truth clustering is often challenging due to data ambiguity and limited supervision. This observation highlights the value of supervisory information, which can guide clustering toward partitions that align with user intent, reduce ambiguity, and improve robustness. Motivated by this, semi-supervised multi-view clustering has been extensively studied [5]–[7].

Among various forms of supervision, pairwise constraints, which indicate whether two samples should belong to the same cluster or not, are particularly attractive due to their low annotation cost and flexibility, and have therefore become a common choice in semi-supervised clustering. Most existing semi-supervised multi-view clustering methods, however, treat such constraints as static: they either assume all constraints are available beforehand or acquire them randomly [8]. This static

usage fails to capture evolving information revealed during iterative clustering and may waste valuable supervisory effort.

To address these limitations, active constraints have been proposed, where the algorithm iteratively identifies informative sample pairs for annotation that indicate whether the two samples belong to the same cluster. The newly acquired constraints are then incorporated to refine the clustering results. By repeating this process, the clustering quality can be iteratively improved with a limited number of annotations. Such human-in-the-loop active clustering has been widely studied in single-view settings, demonstrating its effectiveness in practice.

Extending active constraint selection to multi-view clustering, however, introduces unique challenges. A straightforward solution is to fuse heterogeneous views into a unified representation and perform active selection in this space. Such representation-level fusion, however, may obscure view-specific complementary information and make informativeness estimation vulnerable to noisy or conflicting views, potentially leading to suboptimal constraint selection. Moreover, existing multi-view active clustering methods, such as SMCAC [9] rely on computationally expensive operations like tensor decomposition, making them impractical for interactive annotation scenarios where rapid feedback is essential.

Motivated by these observations, in this paper we investigate scalable multi-view clustering with active constraints, **aiming to efficiently leverage human supervision while preserving complementary information across heterogeneous views.**

Despite its effectiveness, extending active constraint learning to multi-view clustering introduces two key challenges that remain insufficiently addressed in existing work.

Challenge 1: Informative constraint selection . The effectiveness of clustering with active constraints critically depends on selecting sample pairs that can most effectively improve the clustering structure. Ideally, the queried pairs should provide information about ambiguous regions, particularly those near cluster boundaries, where different clustering decisions are most uncertain. However, identifying such informative pairs is non-trivial in multi-view settings, where different views may provide inconsistent or noisy signals for estimating uncertainty and boundary structures.

Challenge 2: Scalability bottleneck in semi-supervised clustering. Many semi-supervised multi-view clustering methods [10] rely on global graph construction, affinity matrix updates, or spectral decomposition. When used in an active-

constraint setting, these costly operations may need to be repeatedly performed during constraint selection and clustering refinement, making human-in-the-loop interaction difficult to scale to large datasets.

To address the challenges of multi-view active clustering, we propose a unified graph-based framework, Triple Graph Active Clustering (**TriGAC**), which models both instance-level structure and inter-view interaction in a consistent graph abstraction. The key idea is that different types of relationships in multi-view active clustering can be naturally decoupled into three graph structures. First, an object graph is constructed within each view to capture instance-level clustering structure via a sparse directed maximum co-cluster probability mechanism. Second, a constraint graph incrementally records user feedback in the form of pairwise must-link relations, forming a dynamic connectivity structure over objects. Third, a view graph is introduced to model inter-view relationships, where nodes correspond to views and edges encode their clustering-level agreement, enabling adaptive view weighting during interaction. Within this unified formulation, the human-in-the-loop clustering process is implemented as an iterative interaction between these graphs. The constraint graph is updated using newly annotated sample pairs and directly modifies the object graphs via local edge rewiring, producing refined per-view partitions. The resulting partitions are then used to update the view graph, which captures evolving inter-view consistency and determines the reliability of each view. These view-level reliabilities, in turn, guide both informative sample selection and cross-view fusion in the next iteration. In this way, **TriGAC** couples constraint acquisition, clustering refinement, and multi-view fusion through interacting graph structures, enabling an efficient and scalable framework for human-in-the-loop multi-view clustering.

The main contributions of this work are summarized as follows:

- We propose **TriGAC**, a unified human-in-the-loop framework that jointly performs active constraint selection and multi-view clustering through three co-evolving lightweight graph structures, without requiring prior knowledge of the cluster number.
- We establish rigorous theoretical guarantees for **TriGAC**, including key structural properties, an upper bound on the number of pairwise constraints required to achieve the optimal clustering, and complexity bounds.
- Extensive experiments on 15 real-world datasets demonstrate that **TriGAC** consistently outperforms state-of-the-art methods in clustering quality, while scaling quasi-linearly in runtime and linearly in memory with dataset size.

Reproducibility. For reproducibility, the source code of **TriGAC** and the complete proofs of all theorems (including the full complexity analysis) are available at <https://github.com/briceloskie/TriGAC>.

II. RELATED WORK

A. How to Use Constraints in Multi-view Clustering

Existing methods incorporate pairwise constraints into multi-view clustering through diverse technical strategies. Tensor-based methods perform t-SVD-based low-rank graph learning with pairwise constraint regularization [9], propagate pairwise constraints through Tucker-decomposition-based subspace clustering [6], or integrate constraints into t-SVD-based tensor representation learning via block-diagonal regularization [11]. NMF-based methods leverage dual-constraint frameworks with hypergraph propagation [12], or enforce inter-view pairwise constraints to align view-specific indicator matrices via non-negative matrix factorization [13]. Graph-based and spectral methods introduce side-constrained graph fusion [14], employ auto-weighted strategies for constrained spectral clustering [15], cast multi-view clustering into matrix completion problems [16], propagate cross-view constraints via low-rank matrix completion [17], perform unified constraint propagation across views [18], or construct bipartite graphs for clustering [10]. Deep learning methods jointly optimize clustering loss, pairwise constraints, and reconstruction via deep autoencoders [19], or adopt progressive self-paced representation learning with cosine similarity constraints [20]. Besides these categories, early representative methods also adopt pairwise constraints through constrained K-Means with co-EM and partial constraint transfer across incomplete mappings [21], or probabilistic mixture models with variational Bayesian inference in subspace projections [22].

TABLE I
TIME AND SPACE COMPLEXITIES OF SEMI-SUPERVISED CLUSTERING WITH CONSTRAINTS

Method	Year	Time Complexity	Space Complexity
UCP [18]	2013	$\mathcal{O}(Tm^2n^2)$	$\mathcal{O}(mn^2)$
MCPCP [17]	2014	$\mathcal{O}(Tn^3)$	$\mathcal{O}(n^2)$
SMVC [22]	2014	$\mathcal{O}(Tmnk\delta)$	$\mathcal{O}(m(nk + \delta))$
MVCC [21]	2014	$\mathcal{O}(Tm(T'nk\delta^2 + n\delta))$	$\mathcal{O}(m(n^2 + \delta^2))$
CMVNMf [13]	2015	$\mathcal{O}(T(mnk + m^2n^2k))$	$\mathcal{O}(m^2n^2 + mnk)$
MVMC [16]	2017	$\mathcal{O}(Tm(n^2k + \delta k^2) + n^3)$	$\mathcal{O}(n^2 + mnk)$
MVSC [15]	2019	$\mathcal{O}(Tn^2(k + \delta))$	$\mathcal{O}(n(m + \delta))$
DMSC [19]	2022	$\mathcal{O}(Tmn(h^2 + d'k))$	$\mathcal{O}(mn(h + d'))$
CTRL [11]	2022	$\mathcal{O}(Tmn^3)$	$\mathcal{O}(mn^2)$
SPDMC [20]	2023	$\mathcal{O}(Tmn(h^2 + d'k))$	$\mathcal{O}(mn(h + d'))$
SCGF [14]	2024	$\mathcal{O}(T(n^3\delta_{CL} + m^2n^2))$	$\mathcal{O}(mn^2 + n\delta_{CL})$
PCBG [10]	2025	$\mathcal{O}(Tna^2\delta_{CL} + TT'na^2)$	$\mathcal{O}(n(a + \delta_{CL}))$
DSNMf [12]	2025	$\mathcal{O}(Tmn^2k)$	$\mathcal{O}(mn^2)$
TSMS [6]	2025	$\mathcal{O}(Tmn^3)$	$\mathcal{O}(mn^2)$
SMCAC [9]	2025	$\mathcal{O}(TT'(mn + n^3) + \delta^3/T^2)$	$\mathcal{O}(mn^2)$
TriGAC	Ours	$\mathcal{O}(Tm^2n + mn \log n + nk(m\Omega + \log k))$	$\mathcal{O}(mn + mk\Omega)$

We summarize the time and space complexities of the aforementioned methods in Table I. Here, T and T' denote the numbers of outer and inner iterations, respectively; m , n , and k denote the numbers of views, samples, clusters, respectively. δ denotes the total number of given pairwise constraints, while δ_{CL} denotes the number of cannot-link constraints. For PCBG, a denotes the number of anchors. For DMSC and SPDMC, h and d' denote the network width and latent embedding dimension, respectively. In our proposed **TriGAC**, Ω is a user-defined constant window size for constraint graph update. Notably, **TriGAC** has low time and space complexity, thereby improving scalability in large-scale semi-supervised multi-view clustering.

B. How to Select Constraints in Clustering

Most existing multi-view semi-supervised clustering studies focus on how to use given constraints, while relatively little attention has been paid to how to actively select informative constraints. Works that jointly consider active constraint selection and multi-view constraint utilization remain scarce, with SMCAC [9] being a notable exception. In contrast, active constraint selection has been more systematically studied in single-view clustering, and existing methods can be roughly divided into two categories [8]: Sample-based methods first identify informative samples and get pairwise constraints via auxiliary structures from them [23]–[27]. Sample-pair-based methods directly evaluate candidate pairs, including density-based strategies [28], [29], hierarchical merging or splitting strategies [30]–[33], and matrix-based criteria based on spectral or similarity perturbation [34], [35].

These active constraint selection strategies are mainly designed for single-view data. When directly applied to multi-view data, a common operation is to first fuse heterogeneous views into a unified representation space and then perform single-view active selection. However, such representation-level fusion may obscure view-specific complementary information, and may also introduce additional computational overhead [36], [37]. This motivates **TriGAC**, which directly performs active constraint selection across views while simultaneously refining the consensus clustering.

III. PROBLEM FORMULATION

DEFINITION 1 (Multi-view Data). *Consider n objects $\{x_1, \dots, x_n\}$. Let $\mathcal{X} = \{\mathbf{X}^{(1)}, \dots, \mathbf{X}^{(m)}\}$ denote a multi-view dataset consisting of m views. The v -th view is represented by a feature matrix $\mathbf{X}^{(v)} \in \mathbb{R}^{n \times d_v}$, where the i -th row corresponds to the representation of object x_i under view v .*

DEFINITION 2. *Pairwise Constraints A pairwise constraint is defined on a pair of objects (x_i, x_j) and can be one of the following:*

- *Must-Link (ML): x_i and x_j should belong to the same cluster;*
- *Cannot-Link (CL): x_i and x_j should belong to different clusters.*

Given the multi-view dataset and pairwise constraints, we consider an interactive clustering setting where constraints are acquired progressively rather than provided beforehand. At each iteration, the algorithm selects object pairs for annotation, obtains pairwise feedback from the annotator, and updates the clustering result accordingly. This process continues until the annotation budget is exhausted.

DEFINITION 3. *Multi-view Clustering with Active Constraints Given a multi-view dataset $\mathcal{X}$ and a query budget B , the goal is to iteratively select informative object pairs for annotation and acquire corresponding pairwise constraints. The acquired pairwise constraints are then utilized to produce a clustering partition $\Pi = \{\pi_1, \dots, \pi_c\}$, where π_i denotes the i -th cluster.*

The objective is to maximize clustering quality while ensuring that the total number of queried constraints does not exceed the budget B .

IV. METHODOLOGY

A. Design Principle and Framework Overview

Graph-Centric Modeling Principle. Multi-view clustering with active constraints involves two fundamental types of relational information: (i) relationships among objects within each view, and (ii) pairwise feedback provided by human annotators. We observe that both types of information are inherently relational and can be naturally represented as edge structures, making graphs a principled modeling choice for this problem.

Specifically, object relationships within each view can be encoded as an object graph, where nodes correspond to samples and edges capture intra-view similarities. This provides a flexible way to represent structural organization under different feature spaces. In addition, human-provided pairwise constraints are inherently edge-based supervision, indicating whether two objects should be connected (must-link) or separated (cannot-link), and can therefore be seamlessly integrated into the same graph structure.

Beyond object-level relationships, multi-view data also exhibits dependencies across views. Instead of performing early fusion in the feature space, we introduce a view graph to explicitly model inter-view relationships, enabling adaptive estimation of view importance and guiding cross-view information integration.

Together, these observations motivate a **unified graph-centric formulation**, where object graphs, a view graph, and a constraint graph jointly represent the system state.

Framework Overview Based on the above principle, we propose a Triple Graph Active Clustering (**TriGAC**).

Algorithm 1 summarizes the overall procedure of **TriGAC**. The input consists of a multi-view dataset $\mathcal{X}$, KNN size K , maximum number of interaction rounds T , representative window size Ω , and query budget B ; the output is the final consensus partition Π . **TriGAC** maintains three co-evolving graph structures: object graphs for preserving view-specific clustering structures, a constraint graph for compactly storing accumulated must-link supervision, and a view graph for estimating adaptive view confidence.

The algorithm first initializes one object graph for each view using the maximum co-cluster probability principle, producing sparse multi-granularity structures that serve as the basis for later selection and refinement (Line 1). Then, in each interaction round, **TriGAC** updates the view graph from the current view-specific and consensus partitions, and obtains the confidence weights $\{\alpha_{v,t}\}_{v=1}^m$ (Line 3). Guided by these weights, it selects a batch of informative objects by combining structural representativeness and dynamic uncertainty across views (Line 4). The selected objects are then sequentially inserted into the constraint graph through pairwise annotations against representative anchor nodes of existing WCCs (Line 5).

Algorithm 1 Framework of TriGAC

Input: Multi-view dataset $\mathcal{X}$, KNN size K , number of iterations T , representative window size Ω , maximum pairwise constraint budget B .

Output: Consensus partition Π

- 1: Initialize object graphs $\{\mathcal{G}_0^{(v)}\}_{v=1}^m$ of each view
 - 2: **for** $t = 1$ to T **do**
 - 3: Build the view graph $\mathcal{G}_{\text{view},t}$ and adaptively update the view confidence weights $\{\alpha_{v,t}\}_{v=1}^m$.
 - 4: Select the most informative batch of objects
 - 5: Sequentially populate constraint graph $\mathcal{G}_{\text{cons},t}$ with pairwise annotations of selected objects
 - 6: Refine object graphs $\{\mathcal{G}_t^{(v)}\}_{v=1}^m$ according to $\mathcal{G}_{\text{cons},t}$ and derive consensus partition Π_t .
 - 7: **end for**
 - 8: **return** final consensus partition Π
-

After the constraint graph is updated, its must-link connectivity is injected into the object graphs to refine view-specific structures and generate the consensus partition Π_t (Line 6). The interaction stops when the query budget B is exhausted or T rounds have been completed, and the final consensus partition is returned.

B. Multi-Granularity Object Graph Construction

The object graph serves as the structural foundation of our framework. Since subsequent operations, including informative pair selection and constraint-driven refinement, are performed on graph structures, it is important that the initialized graph preserves local cluster organization while remaining computationally efficient. To construct the object graphs for all views, we adopt the following assumption.

ASSUMPTION 1. *Objects belonging to the same cluster tend to have higher pairwise similarity than objects from different clusters, resulting in a ranking structure that is consistent with co-cluster probability.*

Based on this assumption, we define a co-cluster probability.

DEFINITION 4 (Co-cluster probability). *Given two objects x_i and x_j , their co-cluster probability in view v is defined as $p_v(x_i, x_j) = \frac{s_v(x_i, x_j)}{1+\epsilon}$, where $s_v(x_i, x_j) = \frac{1}{d(X^{(v)}[i], X^{(v)}[j])+1}$, $d(x_i, x_j)$ denotes the distance between the v -th view of x_i and x_j , and $\epsilon > 0$ is a small constant ensuring numerical stability.*

This formulation preserves the relative ranking of pairwise similarities, enabling robust neighbor selection without requiring calibrated probability estimates. Using the co-cluster probability, TriGAC constructs a directed graph in each view by connecting every object to the object with the maximum co-cluster probability. We refer to this **maximum co-cluster probability graph**.

Empirical analysis reveals that the maximum co-cluster probability graph decomposes into numerous weakly connected components (WCCs), as shown in Fig. 1. Interestingly, samples within the same WCC typically exhibit high semantic

coherence, indicating that WCCs capture meaningful local cluster structures. However, the number of WCCs is often substantially larger than the ground-truth number of clusters, resulting in severe over-partitioning. Therefore, directly treating WCCs as clusters is undesirable.

This observation suggests that WCCs should be viewed as fragmented local structures rather than complete clusters. The challenge is to recover the underlying cluster structure by effectively organizing these fragments while preserving their cluster-relevant information.

To organize WCCs hierarchically, we require a representative structure that can summarize each WCC. We observe that the maximum co-cluster probability graph exhibits a remarkable property.

THEOREM 1 (Unique-Cycle Property). *In the maximum co-cluster probability graph, each weakly connected component contains exactly one directed cycle.*

Proof Sketch. Since every node has exactly one outgoing edge, repeatedly following outgoing edges from any node in a finite WCC must eventually revisit a node, yielding a directed cycle. For uniqueness, suppose two distinct cycles exist in the same WCC. Any node on a cycle must point to the next node on that same cycle; otherwise the cycle would not be closed. Therefore, all outgoing edges of cycle nodes remain within their respective cycles. If the two cycles are disjoint, no directed path can connect them, contradicting the weak connectivity of the component. If they share a node, the unique outgoing edge from that node forces the two cycles to coincide, contradicting the assumption that they are distinct. Thus, each WCC contains exactly one directed cycle. $\square$

Theorem 4 shows that every WCC contains a unique cycle. This cycle can be viewed as the structural core of the corresponding WCC, since every node eventually reaches the cycle by repeatedly following outgoing edges.

THEOREM 2 (Cycle Compactness Property). *Let $\mathcal{L}$ denote a cycle in the maximum co-cluster probability graph, and let $\psi(\cdot)$ denote the outgoing neighbor with the maximum co-cluster probability. Then:*

- (i) **Uniformity.** *For any two cycle nodes $u, v \in \mathcal{L}$, $p(u, \psi(u)) = p(v, \psi(v))$.*
- (ii) **Maximality.** *For any cycle node $u \in \mathcal{L}$ and any off-cycle node $x \notin \mathcal{L}$, $p(u, \psi(u)) \geq p(x, \psi(x))$.*

Proof Sketch. By construction, each node connects to its nearest neighbor. Therefore, along any path formed by repeatedly following outgoing edges, edge lengths are non-increasing. On the unique cycle, this monotonicity becomes circular, forcing all cycle edges to have identical lengths. Since every off-cycle path eventually reaches the cycle, its edge lengths cannot be smaller than the cycle edge length. Because co-cluster probability decreases monotonically with distance, the cycle nodes share identical outgoing edge weights that are no smaller than those of off-cycle nodes. $\square$

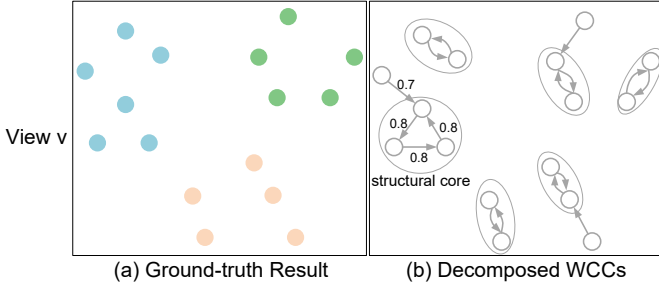


Fig. 1. Illustration of the constructed maximum co-cluster probability graph in view v . (a) The ground-truth partition contains three clusters. (b) The constructed graph decomposes into six WCCs, thereby over-partitioning the data. In each WCC, the unique directed cycle, highlighted by an ellipse, serves as the structural core. The edge weights annotated in one WCC illustrate the structural compactness property: cycle edges have uniform weights and are no smaller than those of off-cycle edges.

The compactness property also admits an intuitive density interpretation. Under a mild radial-cluster assumption, cycle nodes correspond to regions of locally maximal density and therefore form an expected density plateau. This observation further supports the use of cycle nodes as representatives during hierarchical merging.

The above observations motivate a cycle-based merging strategy. Specifically, for each WCC, we select the cycle node with the highest weighted in-degree as its representative agent. Representative agents from different WCCs are then connected according to the maximum co-cluster probability rule, producing a higher-level graph. Since the resulting graph preserves the same structural properties, the process can be recursively repeated until all WCCs are organized into a unified connected graph.

The resulting object graph exhibits a hierarchical multi-granularity structure: lower levels preserve fine-grained local organization, while higher levels progressively capture broader cluster relationships. We will later construct clusters based on evolving these constructed object graphs for all views.

Algorithm 2 presents the Object Graph Initialization procedure. It takes multi-view datasets $X^{(v)}_{v=1}^m$ as input and outputs initialized object graphs $\{\mathcal{G}_0^{(v)}\}_{v=1}^m$. The algorithm begins by constructing an object graph $\mathcal{G}_0^{(v)}$, where the node set is defined as $\mathcal{V}^{(v)} = X^{(v)}$ and no edges are initially included (Line 2). Initially, each node is treated as an agent, forming the agent set A (Line 3). In each iteration, a maximum co-cluster probability graph is first constructed (Line 5), followed by identification of weakly connected components (WCCs) (Line 6). For each WCC, an agent node is selected as the representative, and its outgoing edge is removed from E (Lines 8–14). The resulting refined edges are accumulated across iterations to construct the final object graph.

C. View Graph Construction and Confidence Update

Different views in multi-view clustering often exhibit heterogeneous quality and redundancy. While some views provide complementary information, others may be highly correlated

Algorithm 2 Object Graph Initialization

Input: Multi-view dataset $\mathcal{X}$
Output: Initialized object graphs $\{\mathcal{G}_0^{(v)}\}_{v=1}^m$

```

1: for  $v = 1$  to  $m$  do
2:    $\mathcal{G}_0^{(v)} \leftarrow (\mathcal{V}^{(v)} = X^{(v)}, \mathcal{E}_0^{(v)} = \emptyset)$ 
3:    $A \leftarrow X^{(v)}$ 
4:   while  $|A| > 1$  do
5:      $(A, E) \leftarrow$  Construct maximum co-cluster probability graph.
6:     WCCs  $\leftarrow$  Identify all WCCs in graph  $(A, E)$ 
7:      $A \leftarrow \emptyset$ 
8:     for each WCC  $\in$  WCCs do
9:        $x_a \leftarrow$  select the agent object from WCC
10:      remove the outgoing edge of  $x_a$  from  $E$ 
11:       $A \leftarrow A \cup \{x_a\}$ 
12:    end for
13:     $\mathcal{E}_0^{(v)} \leftarrow \mathcal{E}_0^{(v)} \cup E$ 
14:  end while
15: end for
16: return  $\{\mathcal{G}_0^{(v)}\}_{v=1}^m$ 

```

and therefore contribute redundant evidence. A straightforward strategy that assigns weights purely based on individual agreement with a consensus partition can be suboptimal, as it may assign simultaneously high confidence to mutually similar views while suppressing complementary ones.

To address this issue, we introduce a view graph, which models inter-view relationships in a graph structure. In this graph, each node corresponds to a view, and edges are defined based on the agreement between view-specific partitions. This formulation allows us to explicitly capture redundancy among views rather than treating them independently.

View similarity and graph construction. At iteration t of the interactive clustering process, given the current view-specific partitions $\{\Pi_{t-1}^{(v)}\}_{v=1}^m$, we construct a dynamic view graph to model relationships among views. This construction is performed in every iteration of the interactive loop, ensuring that view dependencies evolve consistently with the clustering state.

The view graph follows the same maximum co-cluster probability principle as the object graph, but operates in the space of views rather than objects. Specifically, we measure the similarity between two views using normalized mutual information (NMI):

$$s(v_i, v_j) = \text{NMI}(\Pi_{t-1}^{(i)}, \Pi_{t-1}^{(j)}), \quad (1)$$

where $\Pi_{t-1}^{(i)}$ denotes the partition obtained from view i at iteration $t - 1$. The corresponding co-cluster probability is defined as

$$p(v_i, v_j) = \frac{s(v_i, v_j)}{1 + \epsilon}. \quad (2)$$

Based on this probability, we construct a directed maximum co-cluster probability graph over views, where each view connects to its most similar counterpart. This graph

naturally decomposes into multiple weakly connected components (WCCs), each grouping views that exhibit high mutual consistency under the current clustering state.

To obtain a more stable representation of inter-view relationships, we further refine this structure following a similar bottom-up merging paradigm as in the object graph initialization (§IV-B), producing a multi-granularity view graph.

Although the merging process produces a unified multi-granularity view graph, not all inter-view connections are equally reliable. Strong connections typically link highly consistent views, whereas weak connections are introduced in later merging stages to bridge different view groups. Therefore, directly treating the unified graph as a single structure would obscure the redundancy patterns among views.

To recover these patterns, we identify a structural breakpoint in the edge-weight distribution. Specifically, we employ the Kneedle algorithm [38] to detect the point where edge strengths exhibit a significant decline. Edges below this breakpoint are removed, resulting in a set of coherent view clusters. This results in L final WCCs $\{C_{\text{view},l}\}_{l=1}^L$.

Each WCC defines a *view cluster*, i.e., a group of mutually consistent views. Importantly, these view clusters are defined over views rather than data objects and are used solely as structural units for confidence allocation, rather than for producing clustering results.

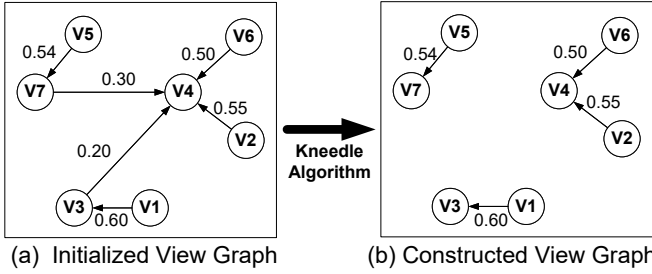


Fig. 2. View graph construction at iteration t . (a) Multi-granularity view graph before pruning. (b) Final view clusters obtained via Kneedle-based pruning, where each cluster groups mutually consistent views.

Hierarchical view weighting. To model both redundancy and reliability among views, we adopt a hierarchical weighting strategy over the induced view clusters. First, each view cluster $C_{\text{view},l}$ is assigned a cluster-level confidence quota based on its average agreement with the current consensus partition:

$$\tilde{\beta}_l = \frac{1}{|C_{\text{view},l}|} \sum_{v \in C_{\text{view},l}} \text{NMI}(\Pi_{t-1}^{(v)}, \Pi_{t-1}), \quad (3)$$

which is normalized as

$$\beta_l = \frac{\tilde{\beta}_l}{\sum_{r=1}^L \tilde{\beta}_r + \epsilon}. \quad (4)$$

Within each cluster, the quota β_l is further distributed to individual views according to their relative reliability:

$$\alpha_{v,t} = \beta_l \cdot \frac{\text{NMI}(\Pi_{t-1}^{(v)}, \Pi_{t-1})}{\sum_{u \in C_{\text{view},l}} \text{NMI}(\Pi_{t-1}^{(u)}, \Pi_{t-1}) + \epsilon}, \quad v \in C_{\text{view},l}. \quad (5)$$

The resulting weight $\alpha_{v,t}$ reflects both (i) redundancy structure captured by the view graph, and (ii) individual reliability of each view with respect to the evolving clustering state. These weights are used in the subsequent iteration for informative pair selection, constraint updating, and consensus refinement.

D. Informative Object Selection

Effective active constraint acquisition aims to select a batch of objects that maximally improves clustering quality under a limited annotation budget. A desirable selection strategy should jointly capture (i) **global structural importance**, which ensures coverage of the data manifold, and (ii) **local boundary ambiguity**, which refines cluster separation. These two signals are complementary: representative objects prevent structural blind spots, while uncertain objects focus annotation effort on decision boundaries where errors are most likely.

Although prior work has attempted to combine these criteria, existing strategies remain limited. Sequential approaches [26] are sensitive to early-stage bias and may accumulate errors as exploration proceeds, while trade-off-based methods [25], [39] often select compromise objects that are neither sufficiently representative nor sufficiently uncertain. These issues suggest that representativeness and uncertainty should not be naively fused at the same level, but instead be organized in a structured, hierarchical manner.

From signal modeling to hierarchical selection. A key observation is that representativeness and uncertainty differ fundamentally in their temporal behavior and semantic role. Representativeness reflects the intrinsic structural importance of objects and can be reliably estimated once from an initial global structure. In contrast, uncertainty depends on evolving cluster assignments and must be updated iteratively. Motivated by this asymmetry, **TriGAC** decouples informative selection into a static structural component and a dynamic refinement mechanism across views to avoid redundancy.

Static representativeness from the initialized graph. We construct an object graph at initialization, which encodes the multi-granularity structure of the data. Since subsequent refinement operations only remove edges without introducing new connections, this initial structure serves as a stable proxy for global data organization.

Based on this property, we define representativeness using the outgoing edge confidence in the initialized graph:

$$\mathcal{H}^{(v)}(x_i) = \begin{cases} 1 - p(x_i, x_j), & \text{if } (x_i, x_j) \in \mathcal{G}_0^{(v)}, \\ 1, & \text{otherwise.} \end{cases} \quad (6)$$

This score is computed once and remains fixed throughout the iterative process.

Dynamic uncertainty from evolving partitions. As constraint information accumulates, the clustering partition $\Pi_t^{(v)}$ is refined progressively. Nodes near cluster boundaries tend to exhibit heterogeneous neighborhood assignments, indicating ambiguity.

We quantify this phenomenon using the normalized entropy of cluster labels within the KNN neighborhood:

Let $\eta = K$ and let ϕ_i^γ denote the fraction of x_i 's K -nearest neighbors that are assigned to cluster γ in view v at iteration t . The dynamic uncertainty is defined as

$$\mathcal{U}_t^{(v)}(x_i) = \frac{-\sum_{\gamma=1}^{\eta} \phi_i^\gamma \log \phi_i^\gamma}{\log \eta}. \quad (7)$$

Unlike representativeness, this score is recomputed at each iteration.

From per-view signals to unified annotation batches. Although both $\mathcal{H}^{(v)}$ and $\mathcal{U}_t^{(v)}$ are defined within each view, the final goal is to select a unified batch of b_t informative objects across all views. This requires resolving two issues: (i) heterogeneity across views, and (ii) redundancy among highly similar views. We therefore adopt a two-stage design: per-view candidate construction followed by cross-view fusion. Let $b_t = \lceil B/T \rceil$ denote the per-iteration annotation budget.

Case 1: Cold-start exploration ($t = 1$). In the initial iteration, no constraints are available, and all nodes are structurally ambiguous in terms of clustering feedback.

In this regime, uncertainty is uninformative, and selection relies solely on representativeness. However, directly pooling top samples across all views would over-represent dominant views and ignore complementary structure. Therefore, **TriGAC** adopts a balanced cross-view sampling strategy: each view contributes high-rank representative nodes in a round-robin fashion until b_t distinct samples are collected.

This ensures that the initial annotation batch provides uniform coverage across heterogeneous views rather than being dominated by any single representation.

Case 2: Iterative refinement ($t > 1$). As constraints accumulate, the clustering structure becomes increasingly informative, and uncertainty emerges near decision boundaries. In this regime, informative selection must balance exploration of uncertain regions with coverage of global structure.

Step 1: Within-view candidate construction. For each view v , we first extract the uncertain set:

$$\mathcal{P}_{\text{unc},t}^{(v)} = \{x_i \mid \mathcal{U}_t^{(v)}(x_i) > 0\}.$$

We prioritize uncertainty because it directly targets regions where the current clustering is least reliable. However, uncertainty alone may be sparse in early iterations; therefore, representativeness is used as a complementary signal to ensure sufficient coverage.

This leads to a prioritized construction:

- If $|\mathcal{P}_{\text{unc},t}^{(v)}| \geq b_t$, we directly select the most uncertain nodes.
- Otherwise, we include all uncertain nodes and supplement them with high-representativeness nodes to preserve structural diversity.

This design ensures that each view contributes a fixed-size candidate pool $\mathcal{P}_t^{(v)}$, avoiding imbalance across views before fusion.

Step 2: Cross-view fusion and global selection. To integrate heterogeneous views fairly, candidate scores are first

normalized within each view-specific pool to remove scale differences, and then weighted by the current view confidence $\alpha_{v,t}$:

$$\hat{s}_i^{(v)} = \frac{s_i^{(v)}}{\max_{x_j \in \mathcal{P}_t^{(v)}} s_j^{(v)} + \epsilon}, \quad \bar{s}_i^{(v)} = \alpha_{v,t} \hat{s}_i^{(v)}, \quad (8)$$

where

$$s_i^{(v)} = \begin{cases} \mathcal{U}_t^{(v)}(x_i), & x_i \in \mathcal{P}_{\text{unc},t}^{(v)}, \\ \mathcal{H}^{(v)}(x_i), & \text{otherwise.} \end{cases} \quad (9)$$

All candidates are then merged across views into a unified pool. We explicitly distinguish uncertainty-driven and representativeness-driven candidates, since they play different roles: the former improves boundary precision, while the latter ensures global coverage.

The final batch $\mathcal{B}_t$ is constructed with a strict priority rule: uncertainty-driven samples are selected first to maximize immediate information gain, and representativeness-driven samples are used only when additional budget remains.

E. Update Constraint Graph with Annotation

Existing approaches for storing pairwise supervision typically adopt either (i) a constraint matrix that explicitly materializes all annotated relations [40], or (ii) transitive closure propagation that expands observed pairwise constraints into a dense supervisory structure [9], [24]. While effective, both strategies incur substantial quadratic memory overhead with respect to the number of samples, making them unsuitable for iterative human-in-the-loop settings.

To address this issue, **TriGAC** maintains a lightweight constraint graph that stores only explicitly acquired must-link relations. Each weakly connected component (WCC) corresponds to a set of samples currently inferred to belong to the same cluster. This design avoids dense matrix construction and eliminates global transitive closure operations, while preserving cluster structure through incremental updates.

From informative samples to constraint updates. At iteration t , the informative node selection module outputs a batch of candidate samples $\mathcal{B}_t$. These samples are the only ones annotated in the current iteration. The goal of constraint updating is to determine, for each $x_i \in \mathcal{B}_t$, whether it should be merged into an existing WCC or form a new singleton component.

Direct comparison between a candidate node and all samples in every WCC is computationally expensive. To enable efficient matching, we approximate each WCC using a small set of representative nodes.

Representative Window. For each WCC $C_{\text{cons},j}$ in view v , we select the top- Ω objects ranked by the fixed representativeness score $\mathcal{H}^{(v)}$, and use them as a compact proxy of the WCC during matching. Since $\mathcal{H}^{(v)}$ is computed once from the initialized object graph and remains unchanged throughout iterations, this representation is stable, and only WCC memberships affect the selected representatives. We refer to the top- Ω objects as the representative window.

Stage I: Anchor construction and ordering. Given a candidate node x_i , we first estimate its affinity to each WCC. For each WCC $C_{\text{cons},j}$, we select an anchor node from its representative window by

$$x_j^* = \arg \min_{x_k \in \bigcup_{v=1}^m \mathcal{R}_j^{(v)}} d_{\text{fus}}(x_i, x_k), \quad (10)$$

where $\mathcal{R}_j^{(v)}$ denotes the representative window of $C_{\text{cons},j}$ in view v , and

$$d_{\text{fus}}(x_i, x_k) = \sum_{v=1}^m \alpha_{v,t} \cdot \hat{d}^{(v)}(x_i, x_k), \quad (11)$$

with $\hat{d}^{(v)}(\cdot, \cdot)$ being the min-max normalized distance in view v , and $\alpha_{v,t}$ the current view confidence.

This produces one anchor per WCC. To prioritize likely matches and enable early termination, anchors are sorted in ascending order of fused distance:

$$\mathcal{Q} = [x_{(1)}^*, x_{(2)}^*, \dots, x_{(M)}^*], \quad (12)$$

where M is the number of existing WCCs. This ordering ensures that structurally closest components are evaluated first, reducing unnecessary interactions.

Stage II: Sequential interaction and graph update. We traverse $\mathcal{Q}$ sequentially. For each anchor $x_{(r)}^*$, the pair $(x_i, x_{(r)}^*)$ is evaluated, and the annotation budget is updated.

If a must-link is returned, x_i is merged into the corresponding WCC by adding edge $(x_i, x_{(r)}^*)$ to $\mathcal{G}_{\text{cons},t}$, and the update for x_i terminates immediately.

If a cannot-link is returned, the current WCC is rejected and the next anchor is examined. If no must-link is obtained after exhausting all anchors, x_i is initialized as a new singleton WCC.

The procedure continues until all samples in $\mathcal{B}_t$ are processed or the annotation budget is exhausted.

Illustration. Figure 3 illustrates an example of constraint graph update. Representative nodes are first selected from each WCC, and candidate node H is compared against these compact summaries under fused distance. The resulting ordered sequence $\mathcal{Q}$ guides sequential interaction, where early termination occurs once a must-link is identified, ensuring efficient constraint acquisition.

Next, we show that the upper bound of pairwise constraints that we need to achieve optimal clustering partition.

THEOREM 3. *Given a multi-view dataset $\mathcal{X} = \{X^{(v)}\}_{v=1}^m$ with n samples and k ground-truth clusters, **TriGAC** requires at most $nk - \frac{k^2}{2} - \frac{k}{2}$ pairwise constraints to achieve the optimal clustering partition.*

Proof. Consider the worst-case scenario where the first k queried nodes each belong to distinct ground-truth clusters. Initially, the constraint graph $\mathcal{G}_{\text{cons},0}$ is empty, so the first queried node initializes the first WCC with zero queries.

For the i -th queried node ($2 \leq i \leq k$), there exist $(i - 1)$ WCCs in $\mathcal{G}_{\text{cons}}$. In the worst case, this node receives a cannot-link (CL) verdict against all existing WCCs, so the

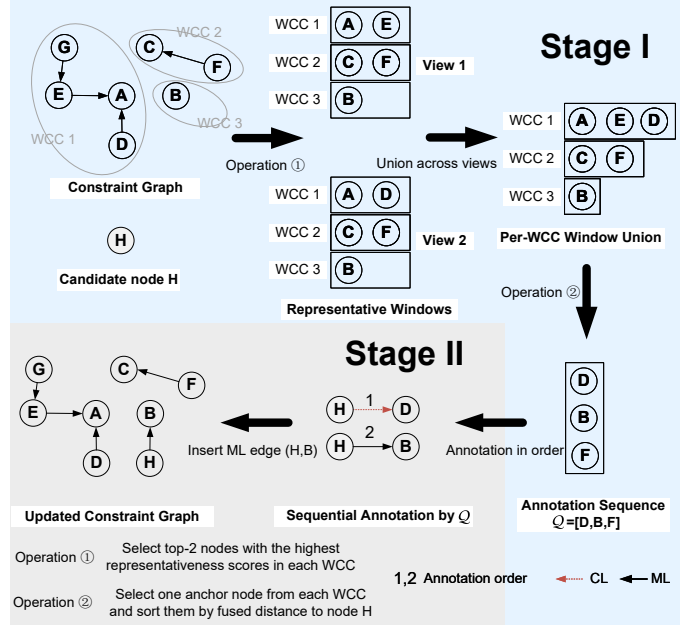


Fig. 3. Illustration of constraint graph updating for a candidate node H .

algorithm exhausts the entire annotation sequence of length $(i - 1)$ and then creates a new singleton WCC. The number of queries consumed is $(i - 1)$. Summing over $i = 2, \dots, k$ yields $\sum_{i=2}^k (i - 1) = \frac{k(k-1)}{2}$.

For each remaining node ($k + 1 \leq i \leq n$), there are exactly k WCCs. Since there are only k ground-truth clusters, this node must share a must-link (ML) with one of them. In the worst case, it receives a CL against the first $(k - 1)$ WCCs and shares an ML only with the last one, consuming k queries. Summing over $i = k + 1, \dots, n$ yields $k(n - k)$.

Therefore, the total number of queries in the worst case is

$$\frac{k(k-1)}{2} + k(n-k) = nk - \frac{k^2}{2} - \frac{k}{2}.$$

□

F. Refine Object Graphs and Consensus Partition Fusion

To incorporate the acquired pairwise supervision into clustering, existing methods typically rely on affinity reconstruction, constraint propagation, or latent representation re-optimization. These approaches repeatedly manipulate dense structures or re-learn embeddings, leading to substantial computational overhead.

In contrast, **TriGAC** exploits a key structural compatibility: both the constraint graph and each view-specific object graph are sparse directed graphs whose weakly connected components (WCCs) encode clustering structure. This enables us to inject must-link constraints through local edge rewiring, avoiding global reconstruction while consistently enforcing supervision within each view.

Object graph refinement. Given the updated constraint graph $\mathcal{G}_{\text{cons},t}$, each view-specific object graph $\mathcal{G}_0^{(v)}$ is refined via local edge rewiring. Recall that in the initialized object graph,

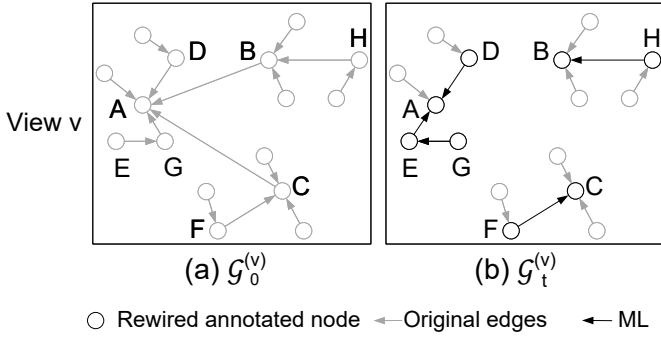


Fig. 4. Illustration of object graph refinement in view v . (a) Initialized object graph $\mathcal{G}_0^{(v)}$, where each node has at most one outgoing edge encoding its maximum co-cluster assignment. (b) Refined object graph $\mathcal{G}_t^{(v)}$ obtained by removing outgoing edges of annotated nodes and injecting must-link edges from $\mathcal{G}_{\text{cons},t}$.

each node has at most one outgoing edge, which represents its maximum co-cluster assignment and induces a WCC-based partition.

For each annotated node $x_i \in \mathcal{V}_{\text{cons},t}$, its outgoing edge is updated according to the constraint information. If x_i participates in a must-link relation in $\mathcal{G}_{\text{cons},t}$, its original outgoing edge is replaced by the corresponding constraint edge; otherwise, its original outgoing edge is removed. Unannotated nodes retain their original structure.

This local rewiring ensures that nodes connected by must-link constraints are forced into the same weakly connected component in the refined graph $\mathcal{G}_t^{(v)}$, while avoiding any global reconstruction. Decomposing $\mathcal{G}_t^{(v)}$ yields the refined per-view partition $\Pi_t^{(v)}$.

Consensus partition fusion. Since all views are refined under the same constraint graph, their WCC structures are aligned through the shared constraint components $\{C_{\text{cons},1}, \dots, C_{\text{cons},M}\}$. This alignment enables sample-level fusion of view-specific clustering results.

Let $\ell_t^{(v)}(x_i)$ denote the cluster assignment of sample x_i in view v . We aggregate evidence from all views via a weighted voting scheme, where each view contributes according to its confidence $\alpha_{v,t}$.

For each sample x_i , its consensus label is determined by:

$$\ell_t(x_i) = \arg \max_{h \in \{1, \dots, M\}} \sum_{v=1}^m \alpha_{v,t} \mathbb{I}(\ell_t^{(v)}(x_i) = h). \quad (13)$$

The final consensus partition is $\Pi_t = \{\pi_h\}_{h=1}^M$, where $\pi_h = \{x_i \mid \ell_t(x_i) = h\}$.

In this way, **TriGAC** integrates structurally consistent refinements from all views while adaptively emphasizing more reliable views through $\alpha_{v,t}$.

G. Complexity Analysis

High-dimensional Preprocessing. **TriGAC** uses Euclidean distance as the default metric. For high-dimensional views, we optionally construct a low-dimensional distance space before graph construction to mitigate the curse of dimensionality [41].

For view v with original dimension D_v , randomized PCA [42] costs $\mathcal{O}(nD_v)$, while UMAP [43] with approximate nearest-neighbor search scales approximately as $\mathcal{O}(n \log n)$. In our implementation, the intermediate PCA dimension and the final UMAP dimension are fixed to 50 and 20, respectively. The total preprocessing cost is therefore

$$\mathcal{O}\left(n \sum_{v=1}^m D_v + mn \log n\right), \quad (14)$$

which preserves the quasi-linear scaling in n when feature dimensions are treated as dataset-dependent constants.

Time Complexity. Let n be the number of samples, m the number of views, T the number of human-in-the-loop iterations, k the number of underlying clusters used only for complexity analysis, and Ω the representative-window size. The KNN local-region construction and object graph initialization require $\mathcal{O}(mn \log n)$ time using efficient nearest-neighbor search. During the iterative refinement process, uncertainty computation, object graph refinement, and consensus partition generation scan the samples across views, resulting in $\mathcal{O}(Tmn)$ time. Updating the constraint graph costs $\mathcal{O}(nk(m\Omega + \log k))$ in the worst case, since each selected candidate node is compared with at most k constraint-graph WCCs and each WCC maintains at most Ω representative candidates per view. Constructing the view graph requires computing pairwise agreement between m per-view partitions, which costs $\mathcal{O}(Tm^2n)$ over all iterations. Therefore, the overall time complexity in the worst case is

$$\mathcal{T} \in \mathcal{O}(Tm^2n + mn \log n + nk(m\Omega + \log k)). \quad (15)$$

When m , T , k , and Ω are treated as small constants, **TriGAC** scales quasi-linearly with the number of samples, i.e., $\mathcal{O}(n \log n)$.

Space Complexity. **TriGAC** mainly stores the sparse graph structures, KNN local regions, representative windows, and auxiliary scores. Since each node in an object graph has at most one outgoing edge, the object graphs, constraint graph, and view graph require only linear storage with respect to the number of involved nodes. Together with the KNN local regions and auxiliary scores, this gives $\mathcal{O}(mn)$ space. The additional representative windows require $\mathcal{O}(mM\Omega)$ space, where M denotes the number of constraint-graph WCCs and Ω is the window size. Since the constraint graph never contains more than k WCCs ($M \leq k$), the worst-case storage is $\mathcal{O}(mk\Omega)$. Excluding raw feature storage, the overall space complexity in the worst case is

$$\mathcal{S} \in \mathcal{O}(mn + mk\Omega), \quad (16)$$

which reduces to $\mathcal{O}(mn)$ when M and Ω are treated as constants.

V. EXPERIMENTS

A. Experimental Setup

The experiments are conducted on a standard PC with a 3.8 GHz CPU (8 cores) and 32 GB of RAM. We use the Adjusted

Rand Index (ARI) [44] to measure the clustering quality. All results are averaged over 10 independent runs.

1) *Datasets*: We evaluate **TriGAC** using 15 real-world datasets spanning diverse domains. To further assess scalability, we utilize the Gaussian mixture model-based generator provided by mvlearn [45] to produce synthetic multi-view datasets with 1 million samples. Their key characteristics are detailed in Table II.

2) *Hyperparameter search protocol*: Following the evaluation protocols in ADP [26] and CSE [33], we perform grid search on small-scale datasets to identify robust hyperparameter settings. Let $q^i(\mathbf{p})$ denote the number of pairwise constraints required to achieve perfect clustering (ARI = 1.0) on dataset i . We first normalize constraint counts within each dataset as: $\theta^i(\mathbf{p}) = \frac{q^i(\mathbf{p})}{\max_{\mathbf{p}'} q^i(\mathbf{p}')}$. We then quantify parameter sensitivity as the average relative degradation from the best configuration:

$$\overline{\text{bias}}(\mathbf{p}) = \frac{1}{n_{\text{ds}}} \sum_{i=1}^{n_{\text{ds}}} \frac{\theta^i(\mathbf{p}) - \min_{\mathbf{p}'} \theta^i(\mathbf{p}')}{\min_{\mathbf{p}'} \theta^i(\mathbf{p}')}. \quad (17)$$

A lower bias indicates that the parameter configuration is closer to the optimal clustering partition across all datasets. We select the best-performing configuration $K = 15$, $T = 20$, and $\Omega = 500$ for all subsequent experiments.

3) *Baselines*:

[ICML'24] **A3S** [31] adopts a cluster-adjustment scheme: it actively selects cluster pairs with the highest expected NMI gain, queries their central samples after a purity test, and refines clustering by aggregating must-linked clusters or splitting impure ones into pure subclusters.

[ICDM'25] **CSE** [33] represents the dataset as a directed sparse graph. It actively queries pairwise constraints between the most uncertain node and representative anchor nodes, progressively refining the graph structure via edge reorganization. [KDD'25] **SMCAC** [9] actively selects informative pairwise constraints via multi-view similarity divergence, and integrates them into a tensor-based multi-view graph learning model to guide discriminative clustering.

[TKDE'25] **PCBG** [10] performs one-step pairwise constrained multi-view clustering via a bipartite graph, encoding must-link constraints with virtual anchors and enforcing cannot-link constraints through a theoretically guaranteed regularization.

[TCYB'25] **TSMSC** [6] jointly performs multi-view subspace clustering and pairwise constraint propagation within a unified tensor low-rank framework, where the learned consensus representation and propagated constraint matrix are optimized together to improve semi-supervised clustering.

[NeurIPS'25] **SparseMVC** [46] learns adaptive multi-view representations by probing cross-view sparsity variations, and fuses them via correlation-informed sample reweighting and cross-view distribution alignment.

[ICDE'25] **RCAGL** [47] performs scalable unsupervised multi-view clustering by jointly learning a consistent anchor graph and view-specific noise components, with a rank constraint enabling one-step clustering without post-processing.

TABLE II
CHARACTERISTICS OF MULTI-VIEW DATASETS.

Datasets	Samples	Clusters	Views	View Dimensions
Small Datasets				
3Sources[48]	169	6	3	[3560, 3631, 3068]
BBCSport[49]	544	5	2	[3183, 3203]
Movies[50]	617	17	2	[1878, 1398]
WebKB[51]	1051	2	2	[1840, 3000]
Cal-5V[52]	1400	7	5	[40, 254, 1984, 512, 928]
RGBD[53]	1499	13	2	[2048, 300]
Youtube[54]	2000	10	6	[2000, 1024, 64, 512, 64, 647]
Cal20[9]	2386	20	6	[48, 40, 254, 1984, 512, 928]
Cora[55]	2708	7	4	[2708, 1433, 2706, 2706]
CiteSeer[55]	3312	6	2	[3312, 3703]
MNIST-4[56]	4000	4	3	[30, 9, 30]
Large Datasets				
MNIST-10K[57]	10000	10	3	[30, 9, 30]
Hdigit[58]	10000	10	2	[784, 256]
Animal[48]	11673	20	4	[2689, 2000, 2001, 2000]
cifar10[59]	50000	10	3	[512, 2048, 1024]
synthetic[45]	1000000	5	5	[5, 5, 5, 5, 5]

Among them, RCAGL and SparseMVC are unsupervised multi-view methods; PCBG, TSMSC, and SMCAC are semi-supervised multi-view methods. A3S and CSE are originally designed for single-view semi-supervised clustering. To adapt A3S and CSE to the multi-view setting, we first employ the fusion module of SparseMVC to generate unified embeddings and then apply their original active constraint-selection and clustering strategies on the fused representations, denoted as A3S-MV and CSE-MV. For all baselines, the hyperparameters are set according to their respective original papers.

B. Clustering Quality

We conduct a comparative experiment to analyze the clustering performance under varying constraint budgets, as shown in Table III and Table IV. SparseMVC and RCAGL are unsupervised methods, A3S cannot generate sufficient constraints due to its inherent logic, so we list the performance of these three methods separately on the far right of the table. Blue cells indicate the highest accuracy among those methods for each constraint budget, while green cells emphasize methods achieving an ARI of 1.000 under a given constraint budget. ‘-’ indicates the baseline failed to return results within 4 hours or exceeded memory limits. SMCAC, PCBG, and TSMSC are not included in Table IV, as they consistently fail to handle the large-scale datasets in our testbed. The result indicates that **TriGAC** outperforms other benchmarks by achieving higher ARI values with the same constraint budget in most cases. Moreover, it is noteworthy that **TriGAC** attains an ARI score of 1 with fewer queries than its competitors, as evidenced in the final row for each dataset. This demonstrates **TriGAC**'s efficacy in reaching optimal clustering results.

We conduct a pairwise t -test to rigorously evaluate the performance of **TriGAC** against its competitors on 11 small datasets at the final constraint budget. The null hypothesis (H_0) posits that there is no difference between the ARI values of **TriGAC** and the others ($R - R^* = 0$). Conversely, the alternative hypothesis (H_a) suggests a significant difference exists ($R - R^* \neq 0$), where R and R^* denote the ARI values for **TriGAC** and the respective baseline algorithms.

TABLE III
CLUSTERING QUALITY V.S. CONSTRAINT BUDGET ON SMALL DATASETS

Dataset	#Cons.	TriGAC	SMCAC	PCBG	TSMSC	CSE-MV	A3S-MV	SparseMVC	RCAGL
3Sources	30	0.5308	0.5657	0.4955	0.5329	0.2665	0.1519	0.3609	0.5641
	60	0.6147	0.5704	0.5726	0.5458	0.3210	0.1893	0.3609	0.5641
	90	0.7719	0.5553	0.6031	0.5476	0.3456	0.1940	0.3609	0.5641
	120	0.8692	0.5832	0.6558	0.5624	0.4707	0.2515	0.3609	0.5641
	150	1.0000	0.5922	0.7096	0.6043	0.5186	0.2515	0.3609	0.5641
BBCSport	60	0.8621	0.7511	0.7545	0.8928	0.1094	0.1215	0.4197	0.5294
	120	0.9131	0.8328	0.8043	0.9017	0.1309	0.1215	0.4197	0.5294
	180	0.9537	0.8491	0.8280	0.9137	0.1543	0.1215	0.4197	0.5294
	240	0.9825	0.8606	0.8389	0.9285	0.1828	0.1215	0.4197	0.5294
	300	1.0000	0.9184	0.8511	0.9413	0.2221	0.1215	0.4197	0.5294
Movies	600	0.1543	0.1294	0.0039	0.1699	0.0457	0.0273	0.0648	0.0678
	1200	0.3020	0.1538	0.0122	0.2992	0.0843	0.0703	0.0648	0.0678
	1800	0.5024	0.1736	0.1152	0.4587	0.1258	0.0775	0.0648	0.0678
	2400	0.8240	0.1635	0.5340	0.5125	0.2369	0.0775	0.0648	0.0678
	3000	1.0000	0.3352	0.7589	0.5596	0.3761	0.0775	0.0648	0.0678
WebKB	240	0.6954	0.9261	0.0011	-0.0070	0.2942	0.0227	0.5849	0.5086
	480	0.8974	0.9523	0.0013	-0.0253	0.3879	0.0227	0.5849	0.5086
	720	0.9706	0.9682	0.0026	0.0339	0.5062	0.0227	0.5849	0.5086
	960	0.9874	0.9706	0.0067	-0.0419	0.6907	0.0227	0.5849	0.5086
	1200	1.0000	0.9743	0.1123	0.0795	0.8675	0.0227	0.5849	0.5086
Cal-5V	300	0.8456	0.5953	0.4503	0.7034	0.6902	0.6359	0.6539	0.7171
	600	0.9219	0.6795	0.6059	0.7083	0.7420	0.6359	0.6539	0.7171
	900	0.9670	0.6594	0.7335	0.7094	0.9212	0.6359	0.6539	0.7171
	1200	0.9967	0.7128	0.7927	0.7181	0.9491	0.6359	0.6539	0.7171
	1500	1.0000	0.7693	0.8230	0.7224	0.9885	0.6359	0.6539	0.7171
RGBD	700	0.3171	0.5248	0.0466	0.2253	0.0958	0.0919	0.2033	0.2729
	1400	0.4691	0.6420	0.0405	0.2867	0.1413	0.0919	0.2033	0.2729
	2100	0.7593	0.6538	0.0675	0.3892	0.2099	0.0919	0.2033	0.2729
	2800	0.9646	0.7077	0.0452	0.4407	0.3011	0.0919	0.2033	0.2729
	3500	1.0000	0.6587	0.0902	0.4791	0.4586	0.0919	0.2033	0.2729
YouTube	1200	0.0893	0.0973	0.0001	0.0836	0.1609	0.1733	0.1976	0.1267
	2400	0.2058	0.1980	0.0098	0.1933	0.2737	0.1733	0.1976	0.1267
	3600	0.4701	0.2054	0.0278	0.6371	0.4360	0.1733	0.1976	0.1267
	4800	0.7707	0.2441	0.1282	0.8946	0.6583	0.1733	0.1976	0.1267
	6000	1.0000	0.2873	0.2713	0.9680	0.8817	0.1733	0.1976	0.1267
Cal20	800	0.6762	0.5672	0.4516	0.3095	0.5625	0.4894	0.5347	0.6073
	1600	0.7924	0.5859	0.6507	0.3717	0.7440	0.4894	0.5347	0.6073
	2400	0.8684	0.6297	0.6222	0.4579	0.7817	0.4894	0.5347	0.6073
	3200	0.9778	0.6568	0.6780	0.4599	0.9206	0.4894	0.5347	0.6073
	4000	1.0000	0.6716	0.8427	0.5412	0.9823	0.4894	0.5347	0.6073
Cora	1000	0.3677	0.2507	0.0997	0.4977	0.0323	0.0042	0.0157	0.1692
	2000	0.5410	0.2597	0.1099	0.5891	0.0795	0.0042	0.0157	0.1692
	3000	0.7840	0.1697	0.1417	0.7441	0.1362	0.0042	0.0157	0.1692
	4000	0.9527	0.5941	0.1992	0.8204	0.1997	0.0042	0.0157	0.1692
	5000	1.0000	0.8397	0.2295	0.8484	0.2920	0.0042	0.0157	0.1692
CiteSeer	1400	0.2152	0.0545	-0.0003	0.3771	0.0314	0.0298	0.0950	0.0593
	2800	0.3709	0.0566	-0.0005	0.4219	0.0889	0.0298	0.0950	0.0593
	4200	0.5951	0.0848	-0.0010	0.5883	0.1739	0.0298	0.0950	0.0593
	5600	0.8707	0.1895	0.0496	0.7602	0.2956	0.0298	0.0950	0.0593
	7000	1.0000	0.4612	0.1439	0.8660	0.4460	0.0298	0.0950	0.0593
MNIST4	900	0.8772	0.8727	0.8014	0.7583	0.7154	0.7603	0.7747	0.7462
	1800	0.9335	0.8806	0.8498	0.7811	0.7933	0.7603	0.7747	0.7462
	2700	0.9841	0.8922	0.8702	0.8112	0.8973	0.7603	0.7747	0.7462
	3600	0.9993	0.9044	0.8972	0.8334	0.9500	0.7603	0.7747	0.7462
	4500	1.0000	0.9354	0.9232	0.8649	0.9842	0.7603	0.7747	0.7462

TABLE IV
CLUSTERING QUALITY V.S. CONSTRAINT BUDGET ON LARGE DATASETS

Dataset	#Cons.	TriGAC	CSE-MV	A3S-MV	SparseMVC	RCAGL
MNIST10k	2000	0.8730	0.8672	0.7914	0.8108	0.5006
	4000	0.9589	0.9389	0.7914	0.8108	0.5006
	6000	0.9897	0.9788	0.7914	0.8108	0.5006
	8000	0.9976	0.9937	0.7914	0.8108	0.5006
	10000	1.0000	0.9987	0.7914	0.8108	0.5006
Hdigit	2200	0.8991	0.8971	0.6651	0.8865	0.7093
	4400	0.9441	0.9140	0.6651	0.8865	0.7093
	6600	0.9799	0.9330	0.6651	0.8865	0.7093
	8800	0.9922	0.9705	0.6651	0.8865	0.7093
	11000	1.0000	1.0000	0.6651	0.8865	0.7093
Animal	12000	0.0785	0.0794	0.0727	0.0870	0.0654
	24000	0.1938	0.1154	0.0727	0.0870	0.0654
	36000	0.4019	0.1021	0.0727	0.0870	0.0654
	48000	0.6567	0.2209	0.0727	0.0870	0.0654
	60000	1.0000	0.4007	0.0727	0.0870	0.0654
cifar10	10000	0.9477	-	-	-	0.9778
	20000	0.9871	-	-	-	0.9778
	30000	0.9977	-	-	-	0.9778
	40000	0.9997	-	-	-	0.9778
	50000	1.0000	-	-	-	0.9778
synthetic	70000	0.9437	-	-	-	-
	140000	0.9825	-	-	-	-
	210000	0.9993	-	-	-	-
	280000	0.9999	-	-	-	-
	350000	1.0000	-	-	-	-

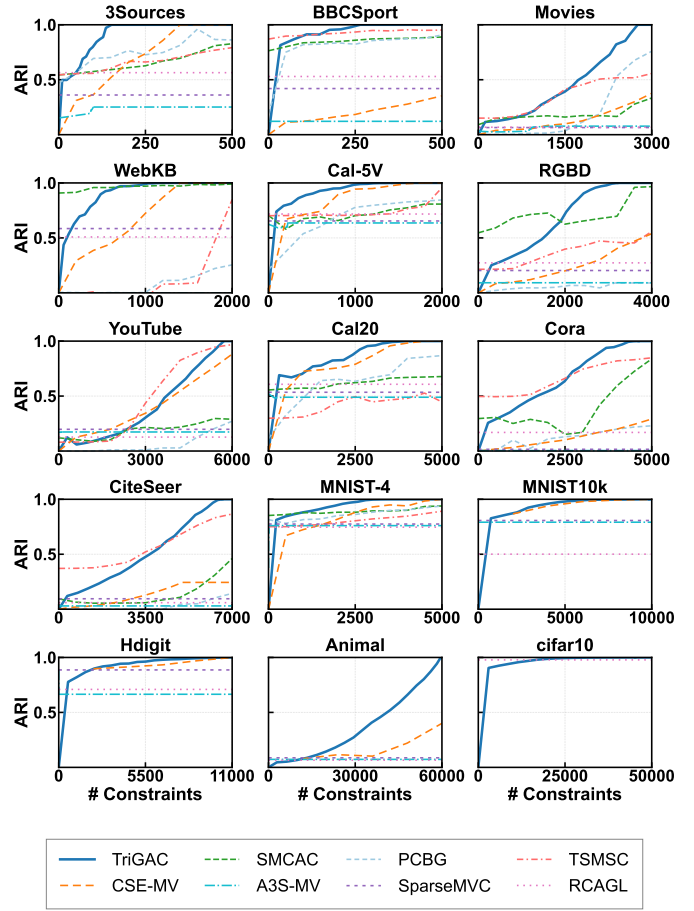


Fig. 5. Performance variation w.r.t. the number of constraints.

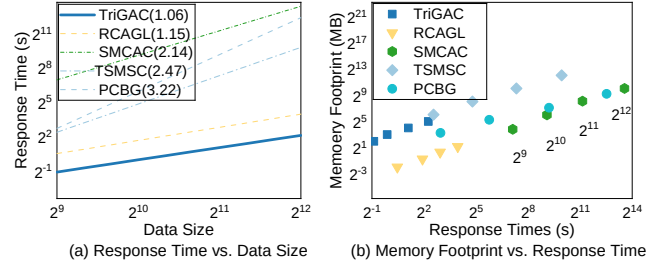


Fig. 6. Scalability evaluation on synthetic datasets of increasing sizes.

As illustrated in Table V, the maximum p -value observed is 2.67×10^{-3} , suggesting that **TriGAC** consistently outperforms the other baselines at a statistically significant level of 1%.

TABLE V
THE RESULT OF PAIRWISE t -TEST.

	SMCAC	PCBG	TSMSC	CSE-MV	A3S-MV	SparseMVC	RCAGL
t -value	4.486853	4.565171	4.044664	3.963261	9.482320	8.266870	7.615573
p -value	1.17×10^{-3}	1.03×10^{-3}	2.34×10^{-3}	2.67×10^{-3}	2.58×10^{-6}	8.82×10^{-6}	1.81×10^{-5}

Additionally, we observe the dynamic performance changes of each algorithm, illustrated in Figure 5. **TriGAC**'s performance exhibits a steady improvement as the number of constraints increases. It is particularly noteworthy that **TriGAC**

generally achieves high-quality clustering results with fewer constraints than the baselines.

C. Computational Efficiency

We evaluate the computational efficiency of **TriGAC** on the synthetic dataset with varying sample sizes. To ensure a consistent annotation ratio across different scales, we set the maximum constraint budget of each semi-supervised clustering method to the dataset size. Figure 6(a) compares the runtime of **TriGAC** with competing methods as the data size increases. On the log-log scale, the runtime curves approximately follow power-law trends, where the fitted slopes reported in the legend reflect the empirical growth rates of different methods. Notably, **TriGAC** exhibits the lowest growth rate among the compared constrained methods, demonstrating the efficiency of the proposed sparse object-graph-based refinement strategy. Although RCAGL achieves competitive computational efficiency, it is an unsupervised clustering method and cannot incorporate pairwise side information into the clustering process.

D. Scalability

The scalability of **TriGAC** is evidenced in the Table IV and the final 4 large-scale datasets in Figure 5. Notably, **TriGAC** can handle a dataset with 1 million data points in 4 hours on a standard PC. In addition, we also measure the memory footprint against response time for varied-sized synthetic datasets, as shown in Figure 6 (b). Dataset sizes are noted next to the data points. **TriGAC** shows better response times and memory footprints, reinforcing **TriGAC**'s scalability.

E. Hyperparameter Analysis

This section analyzes the sensitivity of **TriGAC** to three hyperparameters. Figure 7 presents joint sensitivity heatmaps of (K, T) under different values of Ω , where lighter colors indicate better performance (lower average scaled bias). We observe that $K = 15$ consistently yields optimal performance across all configurations, with the global minimum achieved at $(K = 15, T = 20, \Omega = 500)$. Performance degrades noticeably when K deviates from 15, particularly at $K = 5$ due to insufficient neighborhood information. For T , bias decreases rapidly from 5 to 10 iterations and stabilizes in the range $[15, 25]$. For Ω , substantial improvement occurs from 100 to 200, with diminishing returns beyond 300, indicating that moderate window sizes provide sufficient contextual information. Based on this analysis, we select $K = 15$, $T = 20$, and $\Omega = 500$ as the default configuration, which demonstrates reasonable robustness in the surrounding parameter neighborhood.

F. Structural Property Validation

Table VI empirically supports the over-segmentation phenomenon discussed in Section IV-B. Even the minimum number of WCCs across views substantially exceeds the ground-truth cluster count, indicating that the constructed maximum co-cluster probability graph consistently over-partitions the

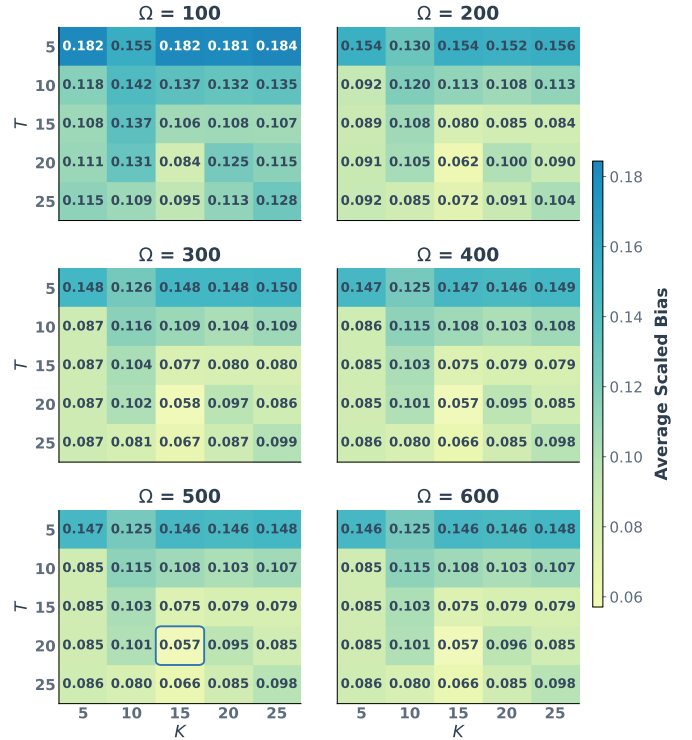


Fig. 7. Joint sensitivity heatmaps of (K, T) under varying Ω .

TABLE VI
OVER-SEGMENTATION STATISTICS OF CONSTRUCTED GRAPH.

Dataset	Clusters	Min WCCs	Avg. WCCs	Dataset	Clusters	Min WCCs	Avg. WCCs
3Sources	6	48	50.7	MNIST-4	4	834	1007.3
BBCSport	5	148	153.0	Youtube	10	528	548.0
Movies	17	165	169.0	MNIST-10K	10	2130	2498.3
Cal-5V	7	375	391.8	Hdigit	10	2741	2748.0
RGBD	13	392	397.5	Animal	20	3089	3136.5
Cora	7	631	673.8	cifar10	10	13166	13365.7
CiteSeer	6	897	903.0	synthetic	5	282786	283354.0

datasets. The small gap between the minimum and average numbers of WCCs further suggests that this structural property is stable across heterogeneous views.

VI. CONCLUSION

In this paper, we propose **TriGAC**, a graph-centric human-in-the-loop framework for multi-view clustering with active constraints. **TriGAC** unifies object graphs, a view graph, and a constraint graph into a co-evolving structure. In the active selection stage, a hierarchical strategy decoupling representativeness and uncertainty is designed to identify informative object pairs for annotation. In the clustering stage, the acquired constraints are injected into each view via local edge rewiring, avoiding costly global reconstruction. We establish theoretical guarantees including an upper bound on the number of pairwise constraints and complexity bounds. Experimental results on 15 real-world datasets demonstrate the effectiveness, scalability, and superiority of **TriGAC**. Future work will explore robust constraint acquisition under noisy annotators, such as LLM-assisted labeling with confidence calibration, to further reduce human supervision cost.

AI-GENERATED CONTENT ACKNOWLEDGEMENT

In several sections of the manuscript, including the Introduction, we used ChatGPT to check spelling errors and refine grammar. In addition, some functions in the experimental code were generated by Deepseek V4 Pro based on our requirements.

REFERENCES

- [1] Y. Li, G. Guo, J. Shi, S. Wang, Q. Li, Efficient integration of multi-view attributed graphs for clustering and embedding, in: 2025 IEEE 41st International Conference on Data Engineering (ICDE), 2025, pp. 3863–3875.
- [2] P. Xu, X. Zhu, D. A. Clifton, Multimodal learning with transformers: A survey, *IEEE Transactions on Pattern Analysis and Machine Intelligence* 45 (10) (2023) 12113–12132.
- [3] A. Moujahid, F. Dornaika, Advanced unsupervised learning: a comprehensive overview of multi-view clustering techniques, *Artificial Intelligence Review* 58 (8) (2025) 234.
- [4] A. R. Chowdhury, A. Gupta, S. Das, Deep multi-view clustering: A comprehensive survey of the contemporary techniques, *Information Fusion* 119 (2025) 103012.
- [5] G. Cui, R. Wang, D. Wu, Y. Li, Semi-supervised multi-view clustering based on nmf with fusion regularization, *ACM Transactions on Knowledge Discovery from Data* 18 (6) (2024) 1–26.
- [6] Z. Yu, C. Yu, K. Yang, X. Chen, Y. Shi, C. P. Chen, Tensor-based semi-supervised multiview subspace clustering with pairwise constraint propagation, *IEEE Transactions on Cybernetics* (2025).
- [7] X. Wang, Y. Wang, Y. Wang, A. Huang, J. Liu, Trusted semi-supervised multi-view classification with contrastive learning, *IEEE Transactions on Multimedia* 26 (2024) 8268–8278.
- [8] C. Xiong, D. M. Johnson, J. J. Corso, Active Clustering with Model-Based Uncertainty Reduction, *IEEE Trans. Pattern Anal. Mach. Intell.* 39 (1) (2016) 5–17.
- [9] C. Zhang, D. Xu, C. Chen, H. Li, Semi-supervised multi-view clustering with active constraints, in: *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 1*, 2025, pp. 1903–1912.
- [10] W. Yu, H. Tao, C. Hou, One-step pairwise constrained multi-view clustering in linear time, *IEEE Transactions on Knowledge and Data Engineering* 37 (9) (2025) 5523–5537.
- [11] Y. Tang, Y. Xie, C. Zhang, W. Zhang, Constrained tensor representation learning for multi-view semi-supervised subspace clustering, *IEEE Transactions on Multimedia* 24 (2022) 3920–3933.
- [12] S. Peng, Z. Huangfu, W. Xie, Z. Yang, F. Nie, Dual constraint based semi-supervised nonnegative matrix factorization for multi-view clustering, *Knowledge-Based Systems* 329 (2025) 114357.
- [13] X. Zhang, L. Zong, X. Liu, H. Yu, Constrained nmf-based multi-view clustering on unmapped data, in: *Proc. AAAI Conf. Artif. Intell.*, AAAI Press, 2015.
- [14] H. Zhang, M. Gong, Y. Gu, F. Nie, X. Li, Side-constrained graph fusion for semi-supervised multi-view clustering, *Neurocomputing* 570 (2024) 127102.
- [15] C. Chen, H. Qian, W. Chen, Z. Zheng, H. Zhu, Auto-weighted multi-view constrained spectral clustering, *Neurocomputing* 366 (2019) 1–11.
- [16] P. Zhao, Y. Jiang, Z.-H. Zhou, Multi-view matrix completion for clustering with side information (2017) 403–415.
- [17] Z. Yang, Y. Hu, H. Liu, H. Chen, Z. Wu, Matrix completion for cross-view pairwise constraint propagation, in: *Proceedings of the 22nd ACM International Conference on Multimedia*, 2014, pp. 897–900.
- [18] Z. Lu, Y. Peng, Unified constraint propagation on multi-view data, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 27, 2013, pp. 640–646.
- [19] R. Chen, Y. Tang, W. Zhang, W. Feng, Deep multi-view semi-supervised clustering with sample pairwise constraints, *Neurocomputing* 500 (2022) 681–693.
- [20] R. Chen, Y. Tang, Y. Xie, W. Feng, W. Zhang, Semisupervised progressive representation learning for deep multiview clustering, *IEEE Transactions on Neural Networks and Learning Systems* 35 (10) (2023) 14341–14355.
- [21] E. Eaton, M. desJardins, S. Jacob, Multi-view constrained clustering with an incomplete mapping between views, *Knowledge and Information Systems* 38 (1) (2014) 231–257.
- [22] S. Günnemann, I. Färber, M. Rüdiger, T. Seidl, Smvc: Semi-supervised multi-view clustering in subspace projections, in: *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ACM, 2014, pp. 253–262.
- [23] S. Basu, A. Banerjee, R. J. Mooney, Active semi-supervision for pairwise constrained clustering, in: *Proceedings of the 2004 SIAM International Conference on Data Mining*, SIAM, 2004, pp. 333–344.
- [24] S. Xiong, J. Azimi, X. Fern, Active learning of constraints for semi-supervised clustering, *IEEE Trans. Knowl. Data Eng.* 26 (1) (2014) 43–54.
- [25] Y. Li, Y. Wang, D.-J. Yu, N. Ye, P. Hu, R. Zhao, ASCENT: Active supervision for semi-supervised learning, *IEEE Trans. Knowl. Data Eng.* 32 (5) (2020) 868–882.
- [26] Y. Shi, Z. Yu, W. Cao, C. L. P. Chen, H.-S. Wong, G. Han, Fast and effective active clustering ensemble based on density peak, *IEEE Trans. Neural Netw. Learn. Syst.* 32 (8) (2021) 3593–3607.
- [27] R. Hazratgholizadeh, M. Balafar, M. Derakhshi, Active constrained deep embedded clustering with dual source, *Appl. Intell.* 53 (5) (2023) 5337–5367.
- [28] A. A. Abin, V.-V. Vu, A density-based approach for querying informative constraints for clustering, *Expert Syst. Appl.* 161 (2020) 113690.
- [29] B. Shen, J. Jiang, F. Qian, D. Li, Y. Ye, G. Ahmadi, Semi-supervised hierarchical ensemble clustering based on an innovative distance metric and constraint information, *Eng. Appl. Artif. Intell.* 124 (2023) 106571.
- [30] T. V. Craenendonck, S. Dumancic, H. Blockeel, COBRA: A fast and simple method for active clustering with pairwise constraints, in: *Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence*, IJCAI-17, Melbourne, Australia, 2017, pp. 2871–2877.
- [31] X. Deng, J. Liu, H. Zhong, F. Feng, C. Shen, X. He, J. Ye, Z. Wang, A3S: A general active clustering method with pairwise constraints, in: *International Conference on Machine Learning*, PMLR, 2024, pp. 10488–105059.
- [32] W.-B. Xie, X. Fu, B. Chen, Y.-L. Lee, T. Deng, T. Zou, X. Wang, Z. Liu, J. Srivastava, Data skeleton learning: Scalable active clustering with sparse graph structures, *Pattern Recognition* (2025) 113018.
- [33] X. Fu, X. Wang, Y. Ni, Cluster skeleton exploration: An effective and scalable active constrained clustering, in: 2025 IEEE International Conference on Data Mining (ICDM), IEEE, 2025, pp. 257–266.
- [34] F. L. Wauthier, N. Jojic, M. I. Jordan, Active spectral clustering via iterative uncertainty reduction, in: *Proceedings of the 18th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, Beijing, China, 2012, p. 1339–1347.
- [35] P. Zhou, B. Sun, X. Liu, L. Du, X. Li, Active clustering ensemble with self-paced learning, *IEEE Trans. Neural Netw. Learn. Syst.* 35 (9) (2024) 12186–12200.
- [36] D. Huang, C.-D. Wang, J.-H. Lai, Fast multi-view clustering via ensembles: Towards scalability, superiority, and simplicity, *IEEE Transactions on Knowledge and Data Engineering* 35 (6) (2023) 5896–5910.
- [37] H. Yuan, Z. Zhang, Q. Guo, L. Chi, S. Ruan, W. Zhou, J. Pang, X. Hao, Dwcl: Dual-weighted contrastive learning for robust multi-view clustering, *Engineering Applications of Artificial Intelligence* 165 (2026) 113532.
- [38] V. Satopaa, J. Albrecht, D. Irwin, B. Raghavan, Finding a” kneedle” in a haystack: Detecting knee points in system behavior, in: 2011 31st international conference on distributed computing systems workshops, IEEE, 2011, pp. 166–171.
- [39] X. Fu, W.-B. Xie, B. Chen, T. Deng, T. Zou, X. Wang, Acdm: An effective and scalable active clustering with pairwise constraint, in: *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*, Boise, Idaho, USA, 2024, pp. 643–652.
- [40] G. González-Almagro, D. Peralta, E. De Poorter, J.-R. Cano, S. García-López, Semi-supervised constrained clustering: an in-depth overview, ranked taxonomy and future research directions, *Artificial Intelligence Review* 58 (157) (2025).
- [41] C. C. Aggarwal, A. Hinneburg, D. A. Keim, On the surprising behavior of distance metrics in high dimensional space, in: *International conference on database theory*, Springer, 2001, pp. 420–434.
- [42] N. Halko, P.-G. Martinsson, J. A. Tropp, Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions, *SIAM review* 53 (2) (2011) 217–288.

- [43] J. Healy, L. McInnes, Uniform manifold approximation and projection, *Nature Reviews Methods Primers* 4 (1) (2024) 82.
- [44] L. Hubert, P. Arabie, Comparing partitions, *Journal of classification* 2 (1985) 193–218.
- [45] R. Perry, G. Mischler, R. Guo, T. Lee, A. Chang, A. Koul, C. Franz, H. Richard, I. Carmichael, P. Ablin, A. Gramfort, J. T. Vogelstein, mvlearn: Multiview machine learning in python, *Journal of Machine Learning Research* 22 (109) (2021) 1–7.
- [46] R. Liu, X. Zou, C. Tang, X. Zheng, X. Hu, K. Sun, X. Liu, Sparsemvc: Probing cross-view sparsity variations for multi-view clustering, in: *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.
- [47] S. Liu, Q. Liao, S. Wang, X. Liu, E. Zhu, Robust and consistent anchor graph learning for multi-view clustering (extended abstract), in: *2025 IEEE 41st International Conference on Data Engineering (ICDE)*, 2025, pp. 4742–4743.
- [48] Z. Xu, H. Liu, C. Lang, T. Wang, Y. Li, M. C. Kampffmeyer, A hubness perspective on representation learning for graph-based multi-view clustering, in: *Proceedings of the Computer Vision and Pattern Recognition Conference*, 2025, pp. 15528–15537.
- [49] M.-S. Chen, C.-D. Wang, J.-H. Lai, Low-rank tensor based proximity learning for multi-view clustering, *IEEE Transactions on Knowledge and Data Engineering* 35 (5) (2022) 5076–5090.
- [50] X. Xie, Y. Xiong, Generalized multi-view learning based on generalized eigenvalues proximal support vector machines, *Expert Systems with Applications* 194 (2022) 116491.
- [51] Q. Qiang, B. Zhang, F. Wang, F. Nie, Fast multi-view discrete clustering with anchor graphs, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 35, 2021, pp. 9360–9367.
- [52] X. Jiang, Z. Ma, Y. Fu, Y. Liao, P. Zhou, Heterogeneity-aware federated deep multi-view clustering towards diverse feature representations, in: *ACM Multimedia 2024*, 2024.
- [53] J. Gao, M. Liu, P. Li, J. Zhang, Z. Chen, Deep multiview adaptive clustering with semantic invariance, *IEEE Transactions on Neural Networks and Learning Systems* 35 (9) (2023) 12965–12978.
- [54] S. Wang, Z. Chen, S. Du, Z. Lin, Learning deep sparse regularizers with applications to multi-view clustering and semi-supervised classification, *IEEE Transactions on Pattern Analysis and Machine Intelligence* 44 (9) (2022) 5042–5055. doi:10.1109/TPAMI.2021.3082632.
- [55] S.-G. Fang, D. Huang, X.-S. Cai, C.-D. Wang, C. He, Y. Tang, Efficient multi-view clustering via unified and discrete bipartite graph learning, *IEEE Transactions on Neural Networks and Learning Systems* 35 (8) (2023) 11436–11447.
- [56] X. Li, H. Zhang, R. Wang, F. Nie, Multiview clustering: A scalable and parameter-free bipartite graph fusion method, *IEEE transactions on pattern analysis and machine intelligence* 44 (1) (2020) 330–344.
- [57] L. Deng, The mnist database of handwritten digit images for machine learning research [best of the web], *IEEE signal processing magazine* 29 (6) (2012) 141–142.
- [58] W. Yan, Y. Zhang, C. Lv, C. Tang, G. Yue, L. Liao, W. Lin, Gcfagg: Global and cross-view feature aggregation for multi-view clustering, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 19863–19872.
- [59] Z. Lu, Y. Yu, L. Ma, F. Nie, R. Wang, Capturing individuality and commonality between anchor graphs for multi-view clustering, in: *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence*, 2025, pp. 5860–5868.
- [60] A. Rodriguez, A. Laio, Clustering by fast search and find of density peaks, *science* 344 (6191) (2014) 1492–1496.
- [61] Q. Huang, A. K. Tung, Lightweight-yet-efficient: Revitalizing ball-tree for point-to-hyperplane nearest neighbor search, in: *2023 IEEE 39th International Conference on Data Engineering (ICDE)*, IEEE, Anaheim, California, USA, 2023, pp. 436–449.

A. Theoretical Analysis

1) Unique-Cycle Property:

THEOREM 4 (Unique-Cycle Property). *In the maximum co-cluster probability graph, each weakly connected component contains exactly one directed cycle.*

Proof. We prove the theorem by contradiction, structured in two parts.

Part 1: Existence of at least one directed cycle in a WCC.

Assume for contradiction that there exists a weakly connected component (WCC) C that contains no directed cycles. Let $n_C \triangleq |C|$ denote the number of objects in C . Since every object in C has exactly one outgoing edge to its maximum co-cluster probability neighbor, we can construct a directed path sequence starting from any object $x_i \in C$:

$$x_i \rightarrow \psi(x_i) \rightarrow \psi^2(x_i) \rightarrow \psi^3(x_i) \rightarrow \dots$$

where $\psi(x_i) \triangleq \arg \max_{x_k \neq x_i} p(x_i, x_k)$ denotes the maximum co-cluster probability neighbor of x_i , and $\psi^t(x_i)$ denotes the object reached by following t outgoing edges starting from x_i :

$$\psi^t(x_i) \triangleq \underbrace{\psi(\psi(\dots \psi(x_i) \dots))}_{t \text{ times}}.$$

Under the assumption that C contains no directed cycles, all objects in the sequence must be distinct.

(I) When $t = 1$, the untraversed object set is $C \setminus \{x_i, \psi(x_i)\}$, the number of traversed objects is 2, and the number of traversed edges is 1.

(II) When $t = n_C - 1$, the untraversed object set is $C \setminus \{x_i, \psi(x_i), \dots, \psi^{n_C-1}(x_i)\} = \emptyset$, the number of traversed objects is n_C , and the number of traversed edges is $n_C - 1$.

(III) When $t = n_C$, since $\psi^{n_C-1}(x_i) \in C$, it must have exactly one outgoing edge. However, the set of untraversed objects is empty. Therefore, $\psi^{n_C}(x_i)$ must be one of the objects already traversed. Let $\psi^{n_C}(x_i) = \psi^j(x_i)$ for some $0 \leq j \leq n_C - 1$. This forms a directed cycle:

$$\psi^j(x_i) \rightarrow \psi^{j+1}(x_i) \rightarrow \dots \rightarrow \psi^{n_C-1}(x_i) \rightarrow \psi^{n_C}(x_i) = \psi^j(x_i)$$

which contradicts the initial assumption that C contains no directed cycles. Therefore, every weakly connected component must contain at least one directed cycle.

Part 2: Uniqueness of the directed cycle in a WCC.

Assume for contradiction that there are two distinct directed cycles in C , denoted by $\mathcal{L}_1$ and $\mathcal{L}_2$. Let $V(\mathcal{L}_1)$ and $V(\mathcal{L}_2)$ be the sets of objects in cycles $\mathcal{L}_1$ and $\mathcal{L}_2$, respectively. We consider two cases based on whether these cycles share objects.

Case 1: $V(\mathcal{L}_1) \cap V(\mathcal{L}_2) = \emptyset$.

Since every object $x_i \in C$ has exactly one outgoing edge, the traversal sequence starting from any object is unique. As shown in Part 1, this path must eventually enter a directed cycle and remain within it. Consider any object $x \in V(\mathcal{L}_1)$, the unique path starting from x is entirely contained within

$\mathcal{L}_1$, meaning $\psi^k(x) \in V(\mathcal{L}_1)$ for all $k \geq 0$. Similarly, for any object $y \in V(\mathcal{L}_2)$, the unique path starting from y is entirely contained within $\mathcal{L}_2$, meaning $\psi^k(y) \in V(\mathcal{L}_2)$ for all $k \geq 0$. Now, consider any object $z \in C$. The path starting from z must eventually enter some directed cycle. If it enters $\mathcal{L}_1$, then all subsequent objects $\psi^k(z)$ (for sufficiently large k) belong to $V(\mathcal{L}_1)$. If it enters $\mathcal{L}_2$, they belong to $V(\mathcal{L}_2)$. Since $V(\mathcal{L}_1)$ and $V(\mathcal{L}_2)$ are disjoint, a path cannot lead to both cycles. Therefore, two node-disjoint directed cycles cannot exist in the same WCC.

Case 2: $\mathcal{L}_1$ and $\mathcal{L}_2$ share at least one object.

Let z be an object such that $z \in V(\mathcal{L}_1)$ and $z \in V(\mathcal{L}_2)$. Since z is in cycle $\mathcal{L}_1$, repeatedly applying the function ψ starting from z must trace the objects of $\mathcal{L}_1$ in sequence and eventually return to z . Let this sequence be:

$$z \rightarrow \psi(z) \rightarrow \dots \rightarrow \psi^{l_1}(z) = z.$$

Similarly, since z is in cycle $\mathcal{L}_2$, repeatedly applying the function ψ starting from z must trace the objects of $\mathcal{L}_2$ in sequence and eventually return to z . Let this sequence be:

$$z \rightarrow \psi(z) \rightarrow \dots \rightarrow \psi^{l_2}(z) = z.$$

Since each object has exactly one outgoing edge, the successor of z , which is $\psi(z)$, is unique. The successor of $\psi(z)$, which is $\psi^2(z)$, is also unique, and so on. Therefore, the sequence generated by starting at z and repeatedly applying ψ is entirely determined and unique. This unique sequence must form the directed cycle containing z . Since this sequence defines both $\mathcal{L}_1$ and $\mathcal{L}_2$, it must be that $\mathcal{L}_1$ and $\mathcal{L}_2$ are the exact same directed cycle. This contradicts the assumption that $\mathcal{L}_1$ and $\mathcal{L}_2$ were distinct directed cycles.

Both cases lead to a contradiction. Therefore, the assumption that a weakly connected component C contains more than one directed cycle is false. Combining the results from both parts, we conclude that every weakly connected component C contains exactly one directed cycle. $\square$

2) Structural Compactness of the Cycle:

LEMMA 1 (Distance Monotonicity). *Given a set of objects $\{x_1, \dots, x_n\}$, if $x_b = \psi(x_a)$ and $x_c = \psi(x_b)$, then*

$$d(x_a, x_b) \geq d(x_b, x_c). \quad (18)$$

Proof. Since $x_c = \psi(x_b)$ is the maximum co-cluster probability neighbor of x_b , and co-cluster probability decreases monotonically with distance, x_c is also the nearest neighbor of x_b in terms of distance. Therefore:

$$d(x_b, x_c) = \min_{x \neq x_b} d(x_b, x).$$

This implies that there does not exist any object x such that $d(x_b, x) < d(x_b, x_c)$, otherwise x_c would not be the maximum co-cluster probability neighbor of x_b . Consequently, we have:

$$d(x_a, x_b) \geq d(x_b, x_c).$$

$\square$

Corollary 1 (Monotonicity along Outgoing Paths). *Given a WCC C in the maximum co-cluster probability graph, for any node $x_i \in C$, consider the path formed by repeatedly following outgoing edges:*

$$x_i \rightarrow \psi(x_i) \rightarrow \psi^2(x_i) \rightarrow \psi^3(x_i) \rightarrow \dots$$

Then the outgoing edge lengths along this path are non-increasing:

$$d(x_i, \psi(x_i)) \geq d(\psi(x_i), \psi^2(x_i)) \geq d(\psi^2(x_i), \psi^3(x_i)) \geq \dots \quad (19)$$

Proof. For any $\tau \geq 0$, consider two consecutive outgoing edges

$$\psi^\tau(x_i) \rightarrow \psi^{\tau+1}(x_i), \quad \psi^{\tau+1}(x_i) \rightarrow \psi^{\tau+2}(x_i).$$

Applying Lemma 1 with $x_a = \psi^\tau(x_i)$, $x_b = \psi^{\tau+1}(x_i)$, and $x_c = \psi^{\tau+2}(x_i)$ yields

$$d(\psi^\tau(x_i), \psi^{\tau+1}(x_i)) \geq d(\psi^{\tau+1}(x_i), \psi^{\tau+2}(x_i)). \quad (20)$$

Chaining this inequality over $\tau = 0, 1, 2, \dots$ establishes the result. $\square$

THEOREM 5 (Cycle Compactness Property). *Let $\mathcal{L}$ denote a cycle in the maximum co-cluster probability graph, and let $\psi(\cdot)$ denote the outgoing neighbor with the maximum co-cluster probability. Then:*

(i) **Uniformity.** *For any two cycle nodes $u, v \in \mathcal{L}$,*

$$p(u, \psi(u)) = p(v, \psi(v)). \quad (21)$$

(ii) **Maximality.** *For any cycle node $u \in \mathcal{L}$ and any off-cycle node $x \notin \mathcal{L}$,*

$$p(u, \psi(u)) \geq p(x, \psi(x)). \quad (22)$$

Proof. Part (i). Consider any node $u_t \in \mathcal{L}$ and index the cycle nodes as

$$u_1 \rightarrow u_2 \rightarrow \dots \rightarrow u_{n_{\mathcal{L}}} \rightarrow u_1,$$

where $u_{t+1} = \psi(u_t)$ for $t = 1, \dots, n_{\mathcal{L}}$ (with $u_{n_{\mathcal{L}}+1} = u_1$). Applying Corollary 1 to the consecutive edges $u_t \rightarrow u_{t+1}$ and $u_{t+1} \rightarrow u_{t+2}$ yields

$$d(u_t, u_{t+1}) \geq d(u_{t+1}, u_{t+2}). \quad (23)$$

Chaining this inequality over all nodes on the cycle, we obtain the circular sequence

$$d(u_1, u_2) \geq \dots \geq d(u_{n_{\mathcal{L}}-1}, u_{n_{\mathcal{L}}}) \geq d(u_{n_{\mathcal{L}}}, u_1) \geq d(u_1, u_2). \quad (24)$$

Since the leftmost and rightmost terms are identical, all intermediate inequalities must hold with equality. Therefore,

$$d(u_1, u_2) = \dots = d(u_{n_{\mathcal{L}}}, u_1) \triangleq d_{\mathcal{L}}. \quad (25)$$

Because co-cluster probability decreases monotonically with distance, we have

$$p(u, \psi(u)) = p(v, \psi(v)), \quad \forall u, v \in \mathcal{L}. \quad (26)$$

Part (ii). For any off-cycle node $x \notin \mathcal{L}$, let its absorption path be

$$x \rightarrow \psi(x) \rightarrow \psi^2(x) \rightarrow \dots \rightarrow \psi^{t-1}(x) \rightarrow u,$$

where $u = \psi^t(x) \in \mathcal{L}$ is the first node on the path that belongs to $\mathcal{L}$. By Corollary 1, the edge lengths along this path are non-increasing:

$$d(x, \psi(x)) \geq \dots \geq d(\psi^{t-1}(x), u) \geq d(u, \psi(u)). \quad (27)$$

Since $u \in \mathcal{L}$, Part (i) implies $d(u, \psi(u)) = d_{\mathcal{L}}$. Thus, by transitivity,

$$d(x, \psi(x)) \geq d_{\mathcal{L}}. \quad (28)$$

Again, because co-cluster probability decreases monotonically with distance, we have

$$p(u, \psi(u)) \geq p(x, \psi(x)), \quad \forall u \in \mathcal{L}, x \notin \mathcal{L}. \quad (29)$$

$\square$

3) **Expected Local-Density Plateau:** Theorem 5 establishes that the unique cycle $\mathcal{L}$ in each WCC is structurally compact: its outgoing edge lengths are uniform and minimal. We now show that, under a radial cluster structure, this structural compactness can be interpreted as a plateau of expected local density.

DEFINITION 5 (Local Density [60]). *For a cutoff distance $d_c > 0$, the local density of object x_i within a WCC C is defined as*

$$\rho_i(d_c) = \sum_{\substack{x_j \in C \\ j \neq i}} \mathbf{1}_{\{d(x_i, x_j) \leq d_c\}}. \quad (30)$$

DEFINITION 6 (Expected Local Density). *Conditioned on the location of object x_i , the expected local density is*

$$\mathbb{E}[\rho_i(d_c) \mid x_i]. \quad (31)$$

ASSUMPTION 2 (Radial Cluster Structure). *The objects in each WCC C are drawn i.i.d. from a probability density $f(\cdot)$ that is continuous, bounded away from zero, and strictly decreasing along any radial direction from its unique mode μ_C .*

LEMMA 2 (Conditional Expectation of Local Density). *Conditioned on x_i , the expectation over the remaining $n_C - 1$ i.i.d. objects admits the closed form*

$$\mathbb{E}[\rho_i(d_c) \mid x_i] = (n_C - 1) \int_{B(x_i, d_c)} f(y) dy, \quad (32)$$

where $B(x_i, d_c)$ denotes the metric ball of radius d_c centered at x_i .

Proof. By linearity of expectation,

$$\mathbb{E}[\rho_i(d_c) \mid x_i] = \sum_{j \neq i} \mathbb{E}[\mathbf{1}_{\{d(x_i, x_j) \leq d_c\}} \mid x_i]. \quad (33)$$

For any $j \neq i$, the conditional expectation of the indicator equals the probability that x_j falls into $B(x_i, d_c)$:

$$\begin{aligned}\mathbb{E}[\mathbf{1}_{\{d(x_i, x_j) \leq d_c\}} | x_i] &= \Pr(d(x_i, x_j) \leq d_c | x_i) \\ &= \int_{B(x_i, d_c)} f(y) dy.\end{aligned}\quad (34)$$

Summing over the $n_C - 1$ remaining i.i.d. objects yields the result. $\square$

Corollary 2 (Expected Local-Density Plateau on the Cycle). *Let C be a weakly connected component (WCC) of the maximum co-cluster probability graph with unique directed cycle $\mathcal{L}$. Under Assumption 2, for any cutoff $d_c > 0$, the cycle nodes form a plateau of expected local density:*

$$\mathbb{E}[\rho_u(d_c) | u] = \mathbb{E}[\rho_v(d_c) | v] \geq \mathbb{E}[\rho_x(d_c) | x], \quad (35)$$

for all $u, v \in \mathcal{L}$ and $x \notin \mathcal{L}$.

Proof. By Theorem 5, the outgoing edge lengths of cycle nodes are uniform and minimal:

$$d(u, \psi(u)) = d(v, \psi(v)) = d_{\mathcal{L}} \leq d(x, \psi(x)), \quad (36)$$

for all $u, v \in \mathcal{L}$ and $x \notin \mathcal{L}$.

Under Assumption 2, the density admits the form $f(y) = g(\|y - \mu_C\|)$ with $g : \mathbb{R}_+ \rightarrow \mathbb{R}_+$ strictly decreasing. Consequently, the nearest-neighbor distance $R_y = d(y, \psi(y))$ serves as an inverse proxy for the radial distance $\|y - \mu_C\|$: a smaller R_y indicates that y lies in a more concentrated region closer to the mode μ_C , and thus $f(y)$ is higher. Therefore, the uniform and minimal edge lengths on the cycle imply that all cycle nodes are at the same radial distance from μ_C and closer than any off-cycle node. Formally,

$$\|u - \mu_C\| = \|v - \mu_C\| \leq \|x - \mu_C\|, \quad (37)$$

for all $u, v \in \mathcal{L}$ and $x \notin \mathcal{L}$.

Because $f(\cdot)$ is strictly decreasing along radial directions, the average density within any metric ball of fixed radius d_c is higher when the ball is centered closer to μ_C . Hence,

$$\int_{B(u, d_c)} f(y) dy = \int_{B(v, d_c)} f(y) dy \geq \int_{B(x, d_c)} f(y) dy. \quad (38)$$

Multiplying by the positive factor $(n_C - 1)$ and applying Lemma 2, we obtain

$$\mathbb{E}[\rho_u(d_c) | u] = \mathbb{E}[\rho_v(d_c) | v] \geq \mathbb{E}[\rho_x(d_c) | x], \quad (39)$$

which completes the proof. $\square$

B. Time Complexity Analysis

TriGAC adopts the Euclidean distance as the default metric. However, Euclidean distance suffers from the curse of dimensionality on high-dimensional features [41]. To handle high-dimensional data without incurring scalability penalties, we apply a lightweight two-stage projection: randomized PCA [42] reduces each view from the original dimension D_v to 50 dimensions in $\mathcal{O}(nD_v)$ time, followed by

UMAP [43] to 20 dimensions via approximate nearest neighbors in $\mathcal{O}(n \log n)$ time. Since D_v , 50, and 20 are constants independent of the sample size n , the overall preprocessing cost is $\mathcal{O}(n \log n)$ and is absorbed by the subsequent $\mathcal{O}(n \log n)$ steps. Consequently, the asymptotic time complexity derived below remains intact.

We now analyze the time cost of each step and summarize the overall complexity.

KNN local region construction. For each view, computing the KNN local regions over n objects can be accomplished in $\mathcal{O}(n \log n)$ time by utilizing the ball-tree technique [61]. With m views, the time complexity is $\mathcal{F}_1 \in \mathcal{O}(mn \log n)$.

Object graph initialization. In each view, the iterative agent-object merging repeatedly queries the maximum co-cluster probability neighbor for every object in the current agent set A . Utilizing the ball-tree, each search costs $\mathcal{O}(\log n)$, and each iteration processes $|A|$ objects. Since $|A|$ is at least halved in every iteration (in the worst case each weakly connected component contains only two objects), the total time per view is bounded by

$$\mathcal{O}\left(n \log n + \frac{n}{2} \log \frac{n}{2} + \dots\right) = \mathcal{O}(n \log n). \quad (40)$$

Thus, with m views, the time complexity is $\mathcal{F}_2 \in \mathcal{O}(mn \log n)$.

Representativeness and uncertainty scores calculation. Computing the representativeness score $\mathcal{H}^{(v)}$ requires a single pass over the n objects in each view, giving $\mathcal{O}(mn)$. For the uncertainty score $\mathcal{U}_t^{(v)}$, the KNN regions are already available; computing the local entropy per object is $\mathcal{O}(K) = \mathcal{O}(1)$. Over T rounds, the time complexity is $\mathcal{F}_3 = \mathcal{O}(Tmn)$.

Informative object selection. In each round, the top- b_t candidates are extracted from m per-view pools. By maintaining a global max-heap for all n objects across views, each extraction and score update costs $\mathcal{O}(\log n)$. Since every object is extracted at most once during the T rounds, the time complexity is $\mathcal{F}_4 = \mathcal{O}(mn \log n)$.

Constraint graph update. For each candidate object x_i , we inspect at most M WCCs in the current constraint graph. Because the graph only grows by merging objects via must-link constraints, M is bounded by the ground-truth cluster number k . For each WCC, we scan at most Ω representative objects across m views to compute the fused distance, which costs $\mathcal{O}(m\Omega)$. Sorting the M anchors takes $\mathcal{O}(M \log M)$. In the worst case, x_i is queried against all M WCCs before a must-link is found or a new singleton WCC is created, adding $\mathcal{O}(M)$. Hence, processing one object costs $\mathcal{O}(k(m\Omega + \log k))$. In the worst case, **TriGAC** processes at most n objects over all T rounds, so the total cost is $\mathcal{F}_5 = \mathcal{O}(nk(m\Omega + \log k))$.

Object graph refinement and consensus fusion. The refinement and consensus fusion (Eq. (??)) scans all m views for every object, costing $\mathcal{O}(mn)$ per round. Over T rounds, the total refinement cost is $\mathcal{F}_6 = \mathcal{O}(Tmn)$.

View graph construction and view confidence update (Line 3 of Algorithm 1). Computing NMI between all pairs of

m per-view partitions takes $\mathcal{O}(m^2n)$ per round because NMI is linear in n for a given partition. Building the view graph and updating the confidence weights over m views costs $\mathcal{O}(m^2)$. Over T rounds, this yields $\mathcal{F}_7 = \mathcal{O}(Tm^2n)$.

Summary. Summing up all steps, in the worst case (when all the objects are selected), the overall time complexity of **TriGAC** is

$$\mathcal{F} = \sum_{i=1}^7 \mathcal{F}_i \in \mathcal{O}(Tm^2n + mn \log n + nk(m\Omega + \log k)). \quad (41)$$

When m , T , k , and Ω are treated as small constants, **TriGAC** scales quasi-linearly with the number of samples, i.e., $\mathcal{O}(n \log n)$.